

Syrian Arab Republic
Ministry of Higher Education and Scientific Research
Manara University — Faculty of Engineering
Robotics and Intelligent Systems



Dynamic Object Grasping Using a 7-DoF Manipulator

Prepared by:

Alaa Hussein

Haidar Saad

Supervised by:

PhD Essa Alghannam

2025/2026

Abstract

This report presents an integrated system for dynamic object grasping with a seven degree-of-freedom Franka Emika Panda manipulator. The system encompasses a complete kinematic and dynamic model derived via screw theory and the Newton–Euler formulation, incorporating space- and body-frame screw matrices and spatial inertia tensors. Due to the computational cost of the Python-based implementation, most runtime kinematics and dynamics calculations were replaced by Pinocchio—a C++-compiled rigid-body dynamics library—enabling real-time performance at the required control timestep.

The motion-planning layer includes two inverse-kinematics solvers—a closed-form analytical solver exploiting the Panda’s seventh-joint redundancy and a Pinocchio-based damped least-squares numerical solver—together with five trajectory profiles: trapezoidal, S-curve, cubic polynomial, quintic polynomial, and the time-optimal TOPP-RA parametrisation.

Three control laws are formulated and implemented: feedback linearisation (computed torque), adaptive sliding mode control, and high-performance real-time model predictive control via ACADOS SQP-RTI. All experiments are conducted in the MuJoCo physics simulator with Pinocchio providing online rigid-body dynamics. The control algorithms were validated first using point stabilisation experiments, then benchmarked on trajectory tracking under nominal, disturbance, and sensor-noise conditions. A comparative benchmark of MPC, SMC, and feedback linearisation on the static grasping problem demonstrates that MPC achieves the best tracking accuracy and lowest control energy, SMC provides the strongest disturbance rejection, and feedback linearisation is the fastest to evaluate.

For the dynamic grasping problem, three algorithms are developed: a forward-scan intercept planner, the CROFT accelerated root-finding search with TOPP-RA refinement, and a contractive MPC that appends an explicit per-step error-contraction constraint. Experimental evaluation shows that CROFT achieves the highest success rate (92.5%) with the lowest grasp-moment alignment error, while contractive MPC provides a fully online alternative without an offline planning phase.

Keywords: dynamic grasping, 7-DoF manipulator, screw theory, inverse kinematics, TOPP-RA, computed torque control, sliding mode control, model predictive control, MuJoCo.

Contents

1	Introduction and Related Work	8
1.1	Motivation	8
1.2	Problem Statement	8
1.3	Related Work	9
1.3.1	Dynamic Grasping	9
1.3.2	Inverse Kinematics for Redundant Manipulators	9
1.3.3	Trajectory Planning	9
1.3.4	Robot Control	9
1.4	Project Objectives	10
1.5	Report Organisation	10
2	Robotic Platform and Simulation Environment	11
2.1	Robotic Platform Selection	11
2.2	Simulation Environment	12
2.2.1	Simulator Selection	12
2.2.2	MuJoCo Configuration	12
3	Kinematic and Dynamic Modelling	15
3.1	Overview	15
3.2	Kinematic Parameters	15
3.2.1	Screw-Theory Framework	15
3.2.2	Space-Frame Screws	16
3.2.3	Body-Frame Screws	16
3.2.4	Home Configuration	16
3.2.5	Velocity Kinematics and Jacobians	17
3.3	Dynamic Parameters	17
3.3.1	Newton–Euler Algorithm with Twists and Wrenches	18
3.4	Actuator and Workspace Limits	19
3.5	Runtime Kinematics and Dynamics: Pinocchio	19
4	Inverse Kinematics and Trajectory Planning	21
4.1	Inverse Kinematics	21
4.1.1	Numerical Damped-Jacobian IK	21

4.1.2	Analytical Closed-Form IK	23
4.2	Trajectory Profiles	24
4.2.1	Overview	24
4.2.2	Trapezoidal Velocity Profile	25
4.2.3	S-Curve (Seven-Stage) Profile	25
4.2.4	Cubic Polynomial Profile	26
4.2.5	Quintic Polynomial Profile	26
4.2.6	Time-Optimal Path Parametrisation (TOPP-RA)	27
4.2.7	Profile Comparison	28
5	Manipulator Control	29
5.1	Feedback Linearisation (Computed Torque)	29
5.1.1	Motivation	29
5.1.2	Control Law	29
5.1.3	Tracking Error Dynamics and Stability	29
5.1.4	Lyapunov Stability Analysis	30
5.2	Adaptive Sliding Mode Control	31
5.2.1	Motivation	31
5.2.2	Sliding Surface	31
5.2.3	Control Law	31
5.2.4	Chattering Reduction via Boundary Layer	31
5.2.5	Adaptive Switching Gains	31
5.2.6	Lyapunov Stability Analysis	32
5.3	Nonlinear Model Predictive Control	32
5.3.1	Receding-Horizon Principle and Advantages	32
5.3.2	Optimal Control Problem Formulation	33
5.3.3	Robot Dynamics and Discretisation	33
5.3.4	Multiple-Shooting Transcription	34
5.3.5	CasADi/IPOPT Implementation	34
5.3.6	ACADOS SQP-RTI Implementation	35
5.4	Gripper Mechanics and Contact Grasping	35
5.4.1	Force Closure and Friction Constraints	35
5.4.2	Two-Phase Gripper Closure	35
5.4.3	Grasp Matrix Analysis	36
6	Experimental Results and Dynamic Grasping	37
6.1	IK Solver Evaluation	37
6.1.1	Experimental Setup	37
6.1.2	Results	38
6.2	Feedback Linearisation Experiments	39
6.2.1	Point-Stabilisation Damping-Ratio Study	39

6.2.2	Trajectory Tracking	43
6.3	Sliding Mode Control Experiments	44
6.3.1	Point Stabilisation	44
6.3.2	Trajectory Tracking with SMC	47
6.4	MPC Point Stabilisation	50
6.4.1	Setup and Results	50
6.4.2	Trajectory Tracking with MPC	52
6.5	Gripper Closure Experiment	53
6.6	Controller Benchmark	54
6.7	Static Grasping	56
6.8	Dynamic Grasping Algorithms	56
6.8.1	Intercept-Point Planning	56
6.8.2	Accelerated Intercept Search (CROFT)	57
6.8.3	Contractive MPC for Dynamic Grasping	59
6.9	Dynamic Grasping Method Comparison	61
6.9.1	Experimental Setup	61
6.9.2	Results	61
6.10	Final Enhancement: Ball-Tracking During Closure	61
	Conclusion and Recommendations	63
	References	65

List of Figures

2.1	The Franka Emika Panda 7-DoF serial manipulator.	12
2.2	MuJoCo simulation environment: Franka Emika Panda manipulator with the spherical target object on the floor plane.	14
4.1	Trapezoidal velocity profile: position $s(t)$, velocity $\dot{s}(t)$, and acceleration $\ddot{s}(t)$	25
4.2	S-curve velocity profile: smooth acceleration transitions eliminate jerk discontinuities.	25
4.3	Cubic polynomial profile: zero velocity at endpoints with non-zero endpoint accelerations.	26
4.4	Quintic polynomial profile: zero velocity and zero acceleration at both endpoints.	27
4.5	TOPP-RA time-optimal velocity profile, saturating both velocity and torque limits simultaneously.	27
5.1	Block diagram of the computed-torque (feedback linearisation) controller. The nonlinear cancellation term $\mathbf{M}\mathbf{v} + \mathbf{C}\dot{\boldsymbol{\theta}} + \mathbf{g}$ transforms the plant into decoupled double integrators driven by the PD auxiliary input.	30
5.2	Block diagram of the adaptive sliding mode controller. The switching term $\mathbf{k}\text{sat}(\mathbf{s}/\Phi)$ drives the state onto the sliding surface $\mathbf{s} = \mathbf{0}$; the adaptive gain law prevents unnecessary chattering.	32
5.3	Franka Hand parallel-jaw gripper and contact geometry on the spherical object.	36
6.1	IK seed-count sweep ($N = 500$ poses per condition). <i>Upper</i> : mean solve time per query (ms). <i>Lower</i> : success rate (%). Blue: analytical solver (frantik); Orange: Pinocchio-DLS numerical solver. Dashed vertical lines mark the 95% success threshold for each solver.	38
6.2	Computed-torque stabilisation, underdamped ($\zeta = 0.5$): per-joint tracking error.	40
6.3	Computed-torque stabilisation, critically damped ($\zeta = 1.0$): per-joint tracking error.	41
6.4	Computed-torque stabilisation, overdamped ($\zeta = 1.5$): per-joint tracking error.	42

6.5	Feedback linearisation trajectory tracking with quintic profile: joint-space error.	43
6.6	Feedback linearisation trajectory tracking with quintic profile: velocity profiles.	43
6.7	Feedback linearisation trajectory tracking with TOPP-RA profile: joint position actual vs desired.	44
6.8	Feedback linearisation trajectory tracking with TOPP-RA profile: tracking error.	44
6.9	SMC per-joint tracking error $e_j(t)$: all joints converge monotonically to below 1 mrad, confirming boundary-layer saturation is active throughout.	45
6.10	SMC sliding surface $s_j(t) = \dot{e}_j + \lambda_j e_j$: zero high-frequency oscillation confirms the saturation function suppresses chattering.	46
6.11	SMC trajectory tracking with quintic profile: per-joint error.	47
6.12	SMC trajectory tracking with quintic profile: actual vs desired velocity.	47
6.13	SMC trajectory tracking with quintic profile: slide surface values.	48
6.14	SMC trajectory tracking with TOPP-RA profile: joint position actual vs desired.	49
6.15	SMC trajectory tracking with TOPP-RA profile: joint velocities actual vs desired.	49
6.16	SMC trajectory tracking with TOPP-RA profile: sliding surface values of the joints.	50
6.17	MPC point stabilisation: per-joint position error $e_j(t) = q_j(t) - q_j^*$	51
6.18	MPC trajectory tracking with quintic profile.	52
6.19	MPC trajectory tracking with TOPP-RA profile.	53
6.20	Gripper closure experiment.	54

List of Tables

2.1	Simulator comparison across key properties.	12
2.2	MuJoCo simulation options.	13
2.3	MuJoCo contact and constraint parameters.	13
2.4	Sphere body and contact parameters.	13
3.1	Franka Emika Panda actuator and workspace limits [14].	19
3.2	Per-joint viscous damping coefficients (N·m·s/rad).	19
3.3	Per-joint position limits (radians).	19
4.1	Comparison of the five trajectory profiles on motion quality, constraint enforcement, and computational requirements.	28
6.1	Operating-point metrics for both IK solvers at the 95% success threshold.	38
6.2	Per-joint CT gains for the three damping-ratio cases ($k_p = \omega_n^2$, $k_d = 2\zeta\omega_n$).	39
6.3	Joint-averaged performance comparison across three damping cases. . .	42
6.4	Per-joint SMC metrics over the full regulation manoeuvre (8.534 s total).	45
6.5	Per-joint MPC stabilisation performance metrics.	52
6.6	Mean tracking RMSE (rad) over 300 seeds, three conditions.	55
6.7	Control effort, computation cost, and joint constraint violations.	55
6.8	Dynamic grasping method comparison (averages over successful trials).	61
6.9	CROFT-Base vs. CROFT-Tracking over 200 trials.	62

Chapter 1

Introduction and Related Work

1.1 Motivation

Dynamic grasping—the problem of intercepting and securely grasping an object that is in motion at the moment of contact—represents one of the most demanding open problems in robotic manipulation. Unlike static grasping, where the target remains stationary and arbitrary amounts of time are available to plan and refine the approach, dynamic grasping imposes simultaneous requirements on speed, precision, and robustness that cannot be satisfied independently. The manipulator must predict where the object will be at a future instant, plan a feasible trajectory to that pose, execute the motion in real time subject to actuator limits, and close the gripper with sufficient force to achieve a stable grasp—all within a window that may be measured in seconds.

The challenge is compounded for redundant manipulators. A seven degree-of-freedom arm offers greater reachability and the ability to satisfy secondary objectives such as joint-limit avoidance or manipulability maximisation through null-space motion, but at the cost of an underspecified inverse-kinematics problem and a larger state space that increases the difficulty of real-time planning and control.

1.2 Problem Statement

This project addresses the following specific problem: given a seven degree-of-freedom Franka Emika Panda manipulator and a spherical object rolling on a planar surface with a known trajectory, design and implement a complete software stack—covering kinematic and dynamic modelling, motion planning, control, and grasping—that can reliably intercept and grasp the moving object within the robot’s reachable workspace.

The stack must operate within the physical constraints of the Panda hardware (joint position, velocity, and torque limits), produce smooth torque-continuous trajectories that do not excite structural vibrations, and handle the redundancy of the seven-joint chain in a principled manner.

1.3 Related Work

1.3.1 Dynamic Grasping

Early approaches to dynamic grasping relied on pre-programmed interception points computed from off-line trajectory prediction combined with open-loop execution [9]. More recent work introduced closed-loop corrections during the approach phase, using visual or force sensing to update the intercept estimate online [10, 11]. The challenge of planning under strict timing constraints has led to the adoption of time-optimal path parametrisation methods [4] and Model Predictive Control formulations that embed constraint satisfaction directly into the optimisation [12].

1.3.2 Inverse Kinematics for Redundant Manipulators

Iterative numerical methods based on the pseudo-inverse Jacobian have been the dominant approach since the foundational work of Whitney [8]. Damped least-squares regularisation, introduced by Nakamura and Hanafusa [7], provides robustness near singularities at the cost of a small task-space error. Analytical closed-form IK exploiting the Panda’s specific kinematic structure—decoupling the wrist from the shoulder through the seventh-joint redundancy parameter—offers microsecond evaluation times essential for repeated IK calls inside a planning loop.

1.3.3 Trajectory Planning

Classical trajectory planning for manipulators generates smooth joint-space profiles satisfying boundary conditions on position and velocity [1]. Polynomial time scalings (cubic, quintic) provide simple closed-form profiles but do not exploit actuator capacity fully. The time-optimal path parametrisation problem, solved robustly by the TOPP-RA algorithm of Pham and Pham [4], produces minimum-time trajectories subject to torque and velocity constraints through a two-pass reachability analysis.

1.3.4 Robot Control

Feedback linearisation (computed torque control) converts the nonlinear manipulator dynamics into a set of decoupled linear integrators, enabling standard PD design [5]. Sliding mode control extends the robustness envelope to bounded model uncertainty and external disturbances through a discontinuous switching term [6], with chattering suppressed by boundary-layer saturation. Model Predictive Control addresses the limitations of both by incorporating explicit constraint handling and a finite-horizon optimisation, with real-time capability achieved through the SQP-RTI scheme implemented in ACADOS [3].

1.4 Project Objectives

The specific objectives of this project are:

1. Extract and validate the complete kinematic and dynamic parameters of the Franka Emika Panda using the screw-theory framework.
2. Implement and benchmark two complementary IK solvers and characterise their success rate and computation time as a function of the number of initialisation seeds.
3. Implement five trajectory profile generators and compare them on motion quality, constraint enforcement, and computational cost.
4. Implement four control laws and benchmark them against each other under nominal, disturbance, and sensor-noise conditions.
5. Design three dynamic grasping algorithms and evaluate their success rate, grasp accuracy, and motion efficiency on a suite of randomised ball-launch trials.

1.5 Report Organisation

The remainder of this report is structured as follows. Chapter 2 describes the robotic platform and simulation environment. Chapter 3 derives the kinematic and dynamic model. Chapter 4 presents the motion-planning layer. Chapter 5 develops the four control laws and gripper mechanics. Chapter 6 reports the experimental results and evaluates the dynamic grasping system. Conclusions and recommendations follow.

Chapter 2

Robotic Platform and Simulation Environment

2.1 Robotic Platform Selection

The Franka Emika Panda, a seven degree-of-freedom (7-DoF) serial manipulator, was selected as the target robotic platform based on the following criteria.

7-DoF kinematic redundancy.

Unlike a standard 6-DoF arm, the additional degree of freedom allows the robot to achieve the same end-effector pose through multiple joint configurations. This flexibility enables joint-limit avoidance, singularity avoidance, and secondary task execution through null-space motion.

Research adoption.

The Franka Panda is among the most widely used platforms in academic robotics research. A large number of reinforcement-learning and motion-planning benchmarks are built around it, and well-maintained simulation models (MJCF/URDF) are publicly available.

Simulation fidelity.

Official and community-maintained simulation models provide accurate inertial parameters, joint limits, and actuator definitions, making simulation results closely representative of real-hardware behaviour.

Moderate complexity.

With seven joints the Panda is sufficiently complex to serve as a meaningful research platform while remaining tractable in terms of state-space dimensionality.



Figure 2.1: The Franka Emika Panda 7-DoF serial manipulator.

2.2 Simulation Environment

2.2.1 Simulator Selection

MuJoCo (Multi-Joint dynamics with Contact) [13] was selected as the primary simulation environment. The selection was driven by its suitability for reinforcement-learning control combined with hardware constraints: while NVIDIA Isaac Sim offers GPU-accelerated parallel simulation, its requirements exceeded the capabilities of the development machines used in this project. Table 2.1 summarises the key properties of the candidate simulators.

Table 2.1: Simulator comparison across key properties.

Feature	MuJoCo	PyBullet	Gazebo	CoppeliaSim	Isaac Sim
Physics engine	Custom	Bullet	ODE/Bullet	Custom	PhysX 5
Simulation speed	Very fast	Fast	Slow	Moderate	Extreme (GPU)
ROS integration	Limited	Limited	Native	Plugin	Native
GPU acceleration	No	No	No	No	Yes
HW requirements	Low-Mod	Low	Moderate	Moderate	Very high
Contact accuracy	Very high	Moderate	Moderate	Moderate	Very high
RL ecosystem	Very strong	Strong	Strong	Moderate	Growing

2.2.2 MuJoCo Configuration

The main simulation options are listed in Table 2.2.

Table 2.2: MuJoCo simulation options.

Parameter	Value	Description
timestep	0.005 s	Simulation timestep (5 ms)
iterations	5	Constraint solver iterations per step
ls_iterations	8	Line-search iterations per solver step
integrator	implicitfast	ODE integrator

Contact and Constraint Parameters

Table 2.3: MuJoCo contact and constraint parameters.

Parameter	Value	Notes
Max contact pairs	12	Simultaneous contact pairs tracked
Floor condim	3	Normal + 2 friction dimensions
Finger solimp	0.95, 0.99, 0.001	Constraint impedance
Finger solref	0.005, 1	Time constant / damping ratio
Gripper solimp	0.99, 0.995, 0.01	Primitive collision impedance
Gripper solref	0.01, 1	Primitive collision reference
Fingertip friction	1 / 0.005 / 0.0001	slide/torsion/roll

Sphere Object Parameters

The grasped object is a solid homogeneous sphere with the parameters listed in Table 2.4. A pair-level contact rule zeroes out all friction between the sphere and the floor, making the sphere slide freely across the ground surface while retaining realistic friction against the gripper pads.

Table 2.4: Sphere body and contact parameters.

Parameter	Value	Notes
Radius	0.03 m	30 mm
Mass	0.1 kg	
Diagonal inertia	$3.6 \times 10^{-5} \text{ kg}\cdot\text{m}^2 (\times 3)$	$I = \frac{2}{5}mr^2$
Joint type	freejoint	6-DoF unconstrained
Body friction (slide/torsion/roll)	1.5 / 0.5 / 0.1	
Floor-pair friction	0 / 0 / 0	Zero friction against floor
Floor-pair condim	1	Normal force only

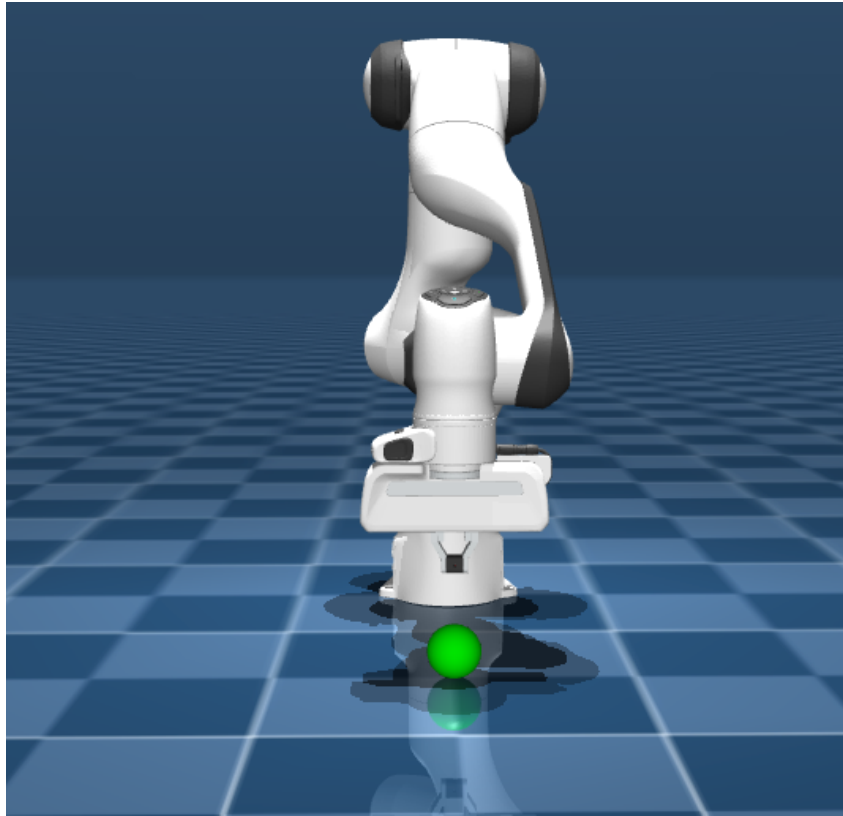


Figure 2.2: MuJoCo simulation environment: Franka Emika Panda manipulator with the spherical target object on the floor plane.

Chapter 3

Kinematic and Dynamic Modelling

3.1 Overview

The first step of the project was the extraction and validation of all kinematic and dynamic parameters of the Franka Emika Panda manipulator. All modelling in this chapter is grounded in screw theory: joint axes are represented as screw axes in both the space and body frames, forward kinematics follows the Product of Exponentials formula, Jacobians are derived from the adjoint action of successive joint transforms, and the dynamic model is built on the Newton–Euler algorithm expressed entirely in terms of spatial velocities (twists) and spatial forces (wrenches). These parameters form the foundation upon which every subsequent computation—forward kinematics, inverse kinematics, dynamics, and control—depends, and were extracted from the official URDF and verified against MuJoCo simulation ground truth. The derivations were initially implemented in Python; however, as discussed at the end of this chapter, the computational cost of the Python implementation necessitated replacing most runtime calculations with the Pinocchio library.

3.2 Kinematic Parameters

3.2.1 Screw-Theory Framework

Screw theory provides a geometrically natural and numerically stable framework for representing serial-manipulator kinematics. A screw axis $\mathbf{S}$ represents combined rotational and translational motion along a joint axis:

$$\mathbf{S} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} \in \mathbb{R}^6, \quad (3.1)$$

where $\boldsymbol{\omega} \in \mathbb{R}^3$ is the unit rotation axis and $\mathbf{v} \in \mathbb{R}^3$ is the linear-velocity component at the origin. For a revolute joint with axis direction $\hat{\boldsymbol{\omega}}$ passing through point $\mathbf{q}$:

$$\mathbf{v} = -\hat{\boldsymbol{\omega}} \times \mathbf{q}. \quad (3.2)$$

The Product of Exponentials (PoE) forward-kinematics formula is:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathbf{S}_1]\theta_1} e^{[\mathbf{S}_2]\theta_2} \dots e^{[\mathbf{S}_n]\theta_n} \mathbf{M}, \quad (3.3)$$

where $\mathbf{M} \in \text{SE}(3)$ is the home configuration and $[\mathbf{S}]$ is the 4×4 matrix representation of a twist:

$$[\mathbf{S}] = \begin{bmatrix} [\boldsymbol{\omega}]_{\times} & \mathbf{v} \\ \mathbf{0}^{\top} & 0 \end{bmatrix}, \quad [\boldsymbol{\omega}]_{\times} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}. \quad (3.4)$$

3.2.2 Space-Frame Screws

The 6×7 space-frame screw matrix $\mathbf{S}_{\text{list}}$ (each column is the screw axis of joint i expressed in the world frame at the home configuration) is:

$$\mathbf{S}_{\text{list}} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & -0.333 & 0 & 0.649 & 0 & 1.033 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.088 \\ 0 & 0 & 0.0825 & 0 & -0.0825 & 0 & 0 \end{pmatrix}. \quad (3.5)$$

3.2.3 Body-Frame Screws

The 6×7 body-frame screw matrix $\mathbf{B}_{\text{list}}$ (each column expressed in the end-effector frame) is:

$$\mathbf{B}_{\text{list}} = \begin{pmatrix} 0 & 0.7071 & 0 & -0.7071 & 0 & -0.7071 & 0 \\ 0 & -0.7071 & 0 & 0.7071 & 0 & 0.7071 & 0 \\ -1 & 0 & -1 & 0 & -1 & 0 & 1 \\ 0.0622 & 0.3486 & 0.0622 & -0.1252 & 0.0622 & 0.1464 & 0 \\ -0.0622 & -0.3486 & -0.0622 & 0.1252 & -0.0622 & -0.1464 & 0 \\ 0 & 0.088 & 0 & -0.0055 & 0 & -0.088 & 0 \end{pmatrix}. \quad (3.6)$$

3.2.4 Home Configuration

The home configuration $\mathbf{M} \in \text{SE}(3)$ (end-effector pose at $\boldsymbol{\theta} = \mathbf{0}$) is:

$$\mathbf{M} = \begin{pmatrix} 0.7071 & 0.7071 & 0 & 0.088 \\ 0.7071 & -0.7071 & 0 & 0 \\ 0 & 0 & -1 & 0.826 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (3.7)$$

3.2.5 Velocity Kinematics and Jacobians

The 6×7 geometric Jacobians map joint velocities $\dot{\boldsymbol{\theta}}$ to the end-effector spatial velocity:

$$\mathcal{V}_b = \mathbf{J}_b(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}}, \quad \mathcal{V}_s = \mathbf{J}_s(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}}, \quad (3.8)$$

where $\mathcal{V}_b$ is the body-frame twist and $\mathcal{V}_s$ is the space-frame twist. The i -th column of the space Jacobian is:

$$\mathbf{J}_{s,i}(\boldsymbol{\theta}) = \text{Ad}_{e^{[\mathbf{S}_1]\theta_1} \dots e^{[\mathbf{S}_{i-1}]\theta_{i-1}}} \mathbf{S}_i. \quad (3.9)$$

Similarly, the i -th column of the body Jacobian is obtained by propagating inward from the end-effector frame:

$$\mathbf{J}_{b,i}(\boldsymbol{\theta}) = \text{Ad}_{e^{-[\mathbf{B}_n]\theta_n} \dots e^{-[\mathbf{B}_{i+1}]\theta_{i+1}}} \mathbf{B}_i. \quad (3.10)$$

Both Jacobians are computed at runtime via Pinocchio and used throughout for IK, velocity matching, and torque mapping.

3.3 Dynamic Parameters

The full manipulator dynamic equation is:

$$\boldsymbol{\tau} = \mathbf{M}(\boldsymbol{\theta}) \ddot{\boldsymbol{\theta}} + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) \dot{\boldsymbol{\theta}} + \mathbf{g}(\boldsymbol{\theta}) + \mathbf{J}^\top(\boldsymbol{\theta}) \mathbf{F}_{\text{tip}} + \mathbf{B} \dot{\boldsymbol{\theta}}, \quad (3.11)$$

where:

- $\boldsymbol{\tau} \in \mathbb{R}^7$: actuator joint torques.
- $\mathbf{M}(\boldsymbol{\theta}) \in \mathbb{R}^{7 \times 7}$: symmetric positive-definite joint-space inertia matrix.
- $\mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) \dot{\boldsymbol{\theta}} \in \mathbb{R}^7$: Coriolis and centripetal torques.
- $\mathbf{g}(\boldsymbol{\theta}) \in \mathbb{R}^7$: gravitational torques.
- $\mathbf{J}^\top \in \mathbb{R}^{7 \times 6}$: Jacobian transpose, mapping external wrench $\mathbf{F}_{\text{tip}}$ to joint torques.
- $\mathbf{B} \in \mathbb{R}^{7 \times 7}$: diagonal joint viscous damping matrix.

The spatial inertia matrix for link i is:

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{I}_i & \mathbf{0} \\ \mathbf{0} & m_i \mathbf{I}_3 \end{bmatrix} \in \mathbb{R}^{6 \times 6}, \quad (3.12)$$

where $\mathbf{I}_i$ is the 3×3 rotational inertia tensor and m_i is the link mass.

3.3.1 Newton–Euler Algorithm with Twists and Wrenches

The dynamic model is evaluated via the Recursive Newton–Euler Algorithm (RNEA), formulated entirely in the screw-theory language of twists and wrenches. The algorithm performs a *forward pass* that propagates spatial velocities and accelerations from the base to the tip, followed by a *backward pass* that propagates spatial forces (wrenches) from the tip back to the base.

Forward pass ($i = 1, \dots, n$). Let $T_{i-1,i}$ denote the transform from frame $i-1$ to frame i , and let $\mathcal{A}_i \in \mathbb{R}^6$ be the screw axis of joint i expressed in frame i . The twist and spatial acceleration of link i are propagated as:

$$\mathcal{V}_i = \text{Ad}_{T_{i-1,i}^{-1}} \mathcal{V}_{i-1} + \mathcal{A}_i \dot{\theta}_i, \quad (3.13)$$

$$\dot{\mathcal{V}}_i = \text{Ad}_{T_{i-1,i}^{-1}} \dot{\mathcal{V}}_{i-1} + [\mathcal{V}_i, \mathcal{A}_i] \dot{\theta}_i + \mathcal{A}_i \ddot{\theta}_i, \quad (3.14)$$

with initial conditions $\mathcal{V}_0 = \mathbf{0}$ and $\dot{\mathcal{V}}_0 = -\mathcal{G}$ (gravity encoded as a spatial acceleration at the fixed base).

Backward pass ($i = n, \dots, 1$). The net wrench required to accelerate link i against its own inertia is:

$$\hat{\mathcal{F}}_i = \mathbf{G}_i \dot{\mathcal{V}}_i - \text{ad}_{\mathcal{V}_i}^* (\mathbf{G}_i \mathcal{V}_i), \quad (3.15)$$

where $\text{ad}_{\mathcal{V}}^*(\cdot)$ is the coadjoint action of the twist $\mathcal{V}$. Adding the wrench transmitted from the distal link gives the total wrench acting at joint i :

$$\mathcal{F}_i = \hat{\mathcal{F}}_i + \text{Ad}_{T_{i,i+1}^{-1}}^\top \mathcal{F}_{i+1}, \quad (3.16)$$

with $\mathcal{F}_{n+1} = \mathbf{F}_{\text{tip}}$. The joint torque is the projection of this wrench onto the joint's screw axis:

$$\tau_i = \mathcal{A}_i^\top \mathcal{F}_i. \quad (3.17)$$

Sweeping equations (3.13)–(3.17) through the kinematic chain recovers the full torque vector $\boldsymbol{\tau}$, consistent with the manipulator equation of motion (3.11).

3.4 Actuator and Workspace Limits

Table 3.1: Franka Emika Panda actuator and workspace limits [14].

Parameter	Symbol	Value
Max joint torques	$\tau_{\max}$	J1–J4: 87 N·m; J5–J7: 12 N·m
Max joint velocities	$\dot{\theta}_{\max}$	J1–J4: 2.175 rad/s; J5–J7: 2.610 rad/s
Rotor armature inertia	$\mathbf{A}$	0.1 kg·m ² (uniform)
Maximum reach	$R_{\max}$	0.855 m
Minimum reach	$R_{\min}$	0.10 m
Body exclusion radius	ρ_{body}	0.12 m

Table 3.2: Per-joint viscous damping coefficients (N·m·s/rad).

Joint	Damping
J1	40
J2	40
J3	40
J4	40
J5	2
J6	2
J7	2

Table 3.3: Per-joint position limits (radians).

Joint	$\theta_{\min}$	$\theta_{\max}$
1	−2.8973	2.8973
2	−1.7628	1.7628
3	−2.8973	2.8973
4	−3.0718	−0.0698
5	−2.8973	2.8973
6	−0.0175	3.7525
7	−2.8973	2.8973

3.5 Runtime Kinematics and Dynamics: Pinocchio

While the screw-theory model derived in this chapter was implemented analytically in Python for validation, the Python implementation proved far too slow for the real-time demands of dynamic grasping. Computing the joint-space inertia matrix alone required approximately 50 ms per step—more than 25 times the 2 ms control timestep—making closed-loop model-based control infeasible.

For all runtime computations we therefore adopted Pinocchio [2], a rigid-body dynamics library written in C++ with efficient Python bindings. Pinocchio takes only the robot’s URDF file as input and automatically constructs the complete kinematic and dynamic model—joint screw axes, link spatial inertias, parent–child transforms, and all associated geometry—without requiring any manual parameter extraction.

Internally, Pinocchio is built on exactly the screw-theory representations described in this chapter. It implements:

- the *Composite Rigid Body Algorithm* (CRBA) for the joint-space inertia matrix $\mathbf{M}(\boldsymbol{\theta})$;
- the *Recursive Newton–Euler Algorithm* (RNEA) for inverse dynamics, delivering $\mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}}$ and $\mathbf{g}(\boldsymbol{\theta})$;
- analytic body and space Jacobians consistent with equations (3.9) and (3.10).

All algorithms exploit the sparse, chain-structured topology of a serial manipulator. The C++ core is exposed to Python with negligible overhead, reducing per-step computation time to 0.1–0.5 ms. Pinocchio was therefore used for all forward-kinematics, Jacobian, and dynamic-quantity evaluations throughout the controllers and planners described in Chapters 4 and 5.

Chapter 4

Inverse Kinematics and Trajectory Planning

4.1 Inverse Kinematics

The inverse-kinematics (IK) problem is to find $\boldsymbol{\theta} \in \mathbb{R}^7$ such that the end-effector achieves a desired pose $\mathbf{T}_d \in \text{SE}(3)$. Because $n = 7 > m = 6$, the system is redundant and no unique closed-form solution exists in general.

4.1.1 Numerical Damped-Jacobian IK

The numerical IK solver is an iterative method based on the inverse Jacobian approach. Starting from a seed joint configuration, the algorithm refines the estimate over up to K iterations: (1) it computes the current end-effector pose via Pinocchio forward kinematics and measures the pose error using the matrix logarithm on $\text{SE}(3)$; (2) it maps the error to a joint correction via the Damped Least-Squares (DLS) pseudo-inverse of the Jacobian, which remains well-conditioned near singularities; and (3) it clips the updated configuration to joint limits. Iterations continue until the angular and translational components of the error fall below the convergence thresholds. To cope with the kinematic redundancy of the 7-DoF chain, the procedure is run from multiple seeds and the valid solution with the smallest proximity score is returned.

Pose Error via the Logarithmic Map

The pose error is computed geometrically on $\text{SE}(3)$ via the matrix logarithmic map:

$$\boldsymbol{\xi} = \left[\log(\mathbf{T}(\boldsymbol{\theta})^{-1} \mathbf{T}_d) \right]^\vee \in \mathbb{R}^6. \quad (4.1)$$

Iteration terminates when $\|\boldsymbol{\omega}\|^2 < 10^{-8}$ and $\|\mathbf{v}\|^2 < 10^{-8}$.

Damped Least-Squares Update

Near singularities the Moore–Penrose pseudo-inverse produces unbounded corrections. Damped Least-Squares (DLS) regularisation minimises:

$$\min_{\Delta \boldsymbol{\theta}} \|\mathbf{J} \Delta \boldsymbol{\theta} - \boldsymbol{\xi}\|^2 + \lambda^2 \|\Delta \boldsymbol{\theta}\|^2, \quad (4.2)$$

yielding:

$$\mathbf{J}_\lambda^+ = \mathbf{J}^\top (\mathbf{J}\mathbf{J}^\top + \lambda^2 \mathbf{I}_6)^{-1}, \quad \lambda^2 = 10^{-4}. \quad (4.3)$$

The joint update is clamped to physical bounds:

$$\boldsymbol{\theta}_{k+1} = \Pi[\boldsymbol{\theta}_k + \mathbf{J}_\lambda^+ \boldsymbol{\xi}_k, \boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}]. \quad (4.4)$$

Redundancy Resolution and Seed Selection

A multi-seed search is used to explore the null space of $\mathbf{J}$ (multiple seeds are evaluated). The best valid solution is selected by the proximity score:

$$\text{score}(\boldsymbol{\theta}) = \max_i \frac{|\theta_i - \theta_{0,i}|}{\dot{\theta}_{\max,i}}, \quad (4.5)$$

This score is proportional to the time the slowest joint would need to travel from the seed position to the solution at its maximum speed. Selecting the solution with the *smallest* score therefore favours the configuration reachable in the least time, improving execution efficiency. Alternatively, the manipulability measure $w(\boldsymbol{\theta}) = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)}$ may be used to favour well-conditioned configurations.

Algorithm 1 Numerical Damped-Jacobian IK (IK_pinocchio)

Require: Target pose $\mathbf{T}_d$, seed $\mathbf{q}_0$, n_{seeds}
Ensure: Best joint configuration $\mathbf{q}^*$ (or None)

- 1: $\varepsilon_\omega^2, \varepsilon_v^2 \leftarrow 10^{-8}; \quad \lambda^2 \leftarrow 10^{-4}; \quad K \leftarrow 25$
- 2: $\text{seeds} \leftarrow [\mathbf{q}_0] + (n_{\text{seeds}} - 1)$ draws from $\mathcal{U}[\mathbf{q}_{\min}, \mathbf{q}_{\max}]$
- 3: $\text{best_score} \leftarrow +\infty; \quad \mathbf{q}^* \leftarrow \text{None}$
- 4: **for** $\mathbf{q}_s$ **in** seeds **do**
- 5: $\mathbf{q} \leftarrow \mathbf{q}_s$
- 6: **for** $k = 1, \dots, K$ **do**
- 7: Compute FK: $\mathbf{T}(\mathbf{q})$ via Pinocchio
- 8: $\boldsymbol{\xi} \leftarrow [\log(\mathbf{T}(\mathbf{q})^{-1}\mathbf{T}_d)]^\vee$
- 9: **if** $\|\boldsymbol{\xi}_{[4:6]}\|^2 < \varepsilon_\omega^2$ **and** $\|\boldsymbol{\xi}_{[1:3]}\|^2 < \varepsilon_v^2$ **then**
- 10: **break** ▷ converged
- 11: **end if**
- 12: $\mathbf{J} \leftarrow \text{frameJacobian}(\mathbf{q})$
- 13: $\Delta\mathbf{q} \leftarrow \mathbf{J}^\top(\mathbf{J}\mathbf{J}^\top + \lambda^2\mathbf{I})^{-1}\boldsymbol{\xi}$ ▷ DLS step
- 14: $\mathbf{q} \leftarrow \text{clip}(\mathbf{q} + \Delta\mathbf{q}, \mathbf{q}_{\min}, \mathbf{q}_{\max})$
- 15: **end for**
- 16: **if** converged **and** $\mathbf{q} \in [\mathbf{q}_{\min}, \mathbf{q}_{\max}]$ **then**
- 17: $\text{score} \leftarrow \max_i |\mathbf{q}_i - \mathbf{q}_{s,i}| / \dot{q}_{\max,i}$
- 18: **if** score $< \text{best_score}$ **then**
- 19: $\text{best_score} \leftarrow \text{score}; \quad \mathbf{q}^* \leftarrow \mathbf{q}$
- 20: **end if**
- 21: **end if**
- 22: **end for**
- 23: **return** $(\mathbf{q}^*, \mathbf{q}^* \neq \text{None})$

4.1.2 Analytical Closed-Form IK

The analytical solver exploits the specific kinematic structure of the Panda to recover joint angles in closed form, without any iterative refinement. The core idea is to treat the seventh joint θ_7 as a free redundancy parameter: for each candidate value of θ_7 , the remaining six joints are uniquely determined through a sequence of geometric sub-problems—decoupling the wrist from the shoulder, applying the law of cosines for the elbow, and using ZYZ Euler-angle decomposition for the shoulder and wrist groups. A discrete sweep over θ_7 is performed and the valid solution with the smallest proximity score is returned.

For the Panda, the seventh joint angle θ_7 is adopted as the scalar redundancy parameter. With θ_7 fixed, the rotation contributed by joints 5–7 is factored out:

$$\mathbf{R}_{0 \rightarrow 3} = \mathbf{R}_d \cdot \mathbf{R}_{4 \rightarrow 7}^\top(\theta_7). \quad (4.6)$$

The elbow angle θ_4 follows from the law of cosines applied to the triangle $\triangle O_2 O_4 O_6$:

$$\theta_4 = \arccos\left(\frac{\|O_2 O_4\|^2 + \|O_4 O_6\|^2 - \|O_2 O_6\|^2}{2 \|O_2 O_4\| \|O_4 O_6\|}\right). \quad (4.7)$$

The shoulder joints are recovered by ZYZ Euler-angle decomposition of $\mathbf{R}_{0 \rightarrow 3}$:

$$\theta_2 = \arctan 2\left(\sqrt{R_{02}^2 + R_{12}^2}, R_{22}\right), \quad (4.8)$$

$$\theta_1 = \arctan 2(R_{12}, R_{02}), \quad \theta_3 = \arctan 2(R_{21}, -R_{20}). \quad (4.9)$$

The wrist joints θ_5, θ_6 are recovered analogously from the residual rotation $\mathbf{R}_{4 \rightarrow 6}$. A sweep of discrete values of θ_7 is evaluated; the candidate minimising Eq. (4.5) is returned.

Algorithm 2 Analytical Closed-Form IK (IK_frantik)

Require: Target pose $\mathbf{T}$, seed $\mathbf{q}_0, n_{\text{seeds}}$

Ensure: Best joint configuration $\mathbf{q}^*$ (or None)

```

1: best_q  $\leftarrow$  None; best_score  $\leftarrow$   $+\infty$ 
2:  $\mathbf{q}_{\text{sol}} \leftarrow \text{CC\_IK}(\mathbf{T}, q_{0,7}, \mathbf{q}_0)$   $\triangleright$  seed 0: use current  $q_7$ 
3: if  $\mathbf{q}_{\text{sol}}$  finite and  $\in [q_{\min}, q_{\max}]$  then
4:   best_q  $\leftarrow \mathbf{q}_{\text{sol}}$ ; best_score  $\leftarrow \max_i |q_{\text{sol},i} - q_{0,i}| / \dot{q}_{\max,i}$ 
5: end if
6: for  $\mathbf{q}_{\text{extra}}$  in  $(n_{\text{seeds}} - 1)$  draws from  $\mathcal{U}[q_{\min}, q_{\max}]$  do
7:    $\mathbf{q}_{\text{sol}} \leftarrow \text{CC\_IK}(\mathbf{T}, q_{\text{extra},7}, \mathbf{q}_{\text{extra}})$   $\triangleright$  CC_IK: the IK_frantik closed-form solver
8:   if not (finite and  $\in [q_{\min}, q_{\max}]$ ) then continue
9:   end if
10:  score  $\leftarrow \max_i |q_{\text{sol},i} - q_{\text{extra},i}| / \dot{q}_{\max,i}$ 
11:  if score < best_score then
12:    best_score  $\leftarrow$  score; best_q  $\leftarrow \mathbf{q}_{\text{sol}}$ 
13:  end if
14: end for
15: return (best_q, best_q  $\neq$  None)

```

4.2 Trajectory Profiles

4.2.1 Overview

All trajectories are formulated in joint space rather than in task (Cartesian) space. Joint-space trajectories are preferred for three reasons: (i) they remain entirely within the kinematically feasible joint range by construction, without requiring intermediate workspace checks; (ii) they avoid the Cartesian singularities that can arise when interpolating directly in SE(3); and (iii) they trivially enforce the per-joint velocity and torque bounds imposed by the Panda's hardware.

The joint trajectory is recovered from the normalised path parameter $s(t) \in [0, 1]$:

$$\theta(t) = \theta_{\text{start}} + s(t) (\theta_{\text{end}} - \theta_{\text{start}}). \quad (4.10)$$

4.2.2 Trapezoidal Velocity Profile

The trapezoidal profile has three phases: constant-acceleration ramp, constant-velocity coast, and symmetric deceleration. The per-joint constraints:

$$|\Delta\theta_i| \cdot v \leq \dot{\theta}_{\text{max},i}, \quad |\Delta\theta_i| \cdot a \leq \ddot{\theta}_{\text{max},i}, \quad (4.11)$$

yield the minimum traversal time $T = (a + v^2)/(va)$. If $v^2/a > 1$ the coast phase vanishes (bang-bang profile).

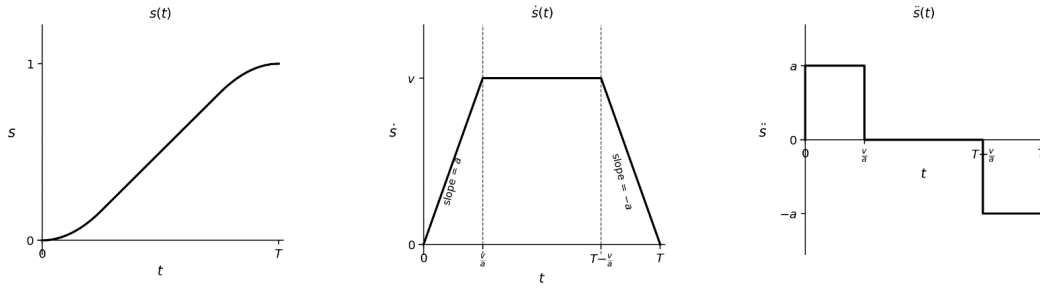


Figure 4.1: Trapezoidal velocity profile: position $s(t)$, velocity $\dot{s}(t)$, and acceleration $\ddot{s}(t)$.

4.2.3 S-Curve (Seven-Stage) Profile

The S-curve introduces a bounded jerk $J = d^3s/dt^3$, producing seven stages symmetric about the midpoint. Stages (1)–(3) form the ramp-up (constant positive jerk, constant acceleration, constant negative jerk); stage (4) is the constant-velocity coast; stages (5)–(7) mirror the ramp-up. Compared with the trapezoidal profile, the S-curve requires $\approx 3\text{--}5\%$ longer motion time but eliminates instantaneous acceleration discontinuities.

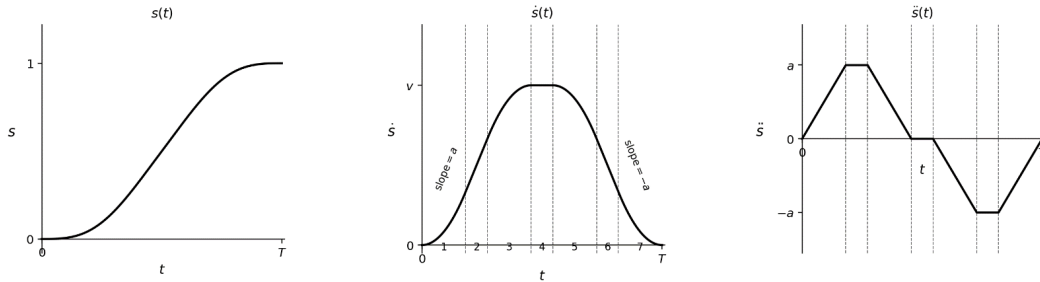


Figure 4.2: S-curve velocity profile: smooth acceleration transitions eliminate jerk discontinuities.

4.2.4 Cubic Polynomial Profile

The cubic time scaling:

$$s(t) = a_0 + a_1t + a_2t^2 + a_3t^3, \quad (4.12)$$

with boundary conditions $s(0) = \dot{s}(0) = 0$, $s(T) = 1$, $\dot{s}(T) = 0$ uniquely yields:

$$a_0 = a_1 = 0, \quad a_2 = \frac{3}{T^2}, \quad a_3 = -\frac{2}{T^3}. \quad (4.13)$$

Peak velocity: $\dot{s}_{\max} = 3/(2T)$. Peak acceleration at endpoints: $|\ddot{s}|_{\max} = 6/T^2$ (non-zero, causing infinite-jerk impulses at $t = 0$ and $t = T$).

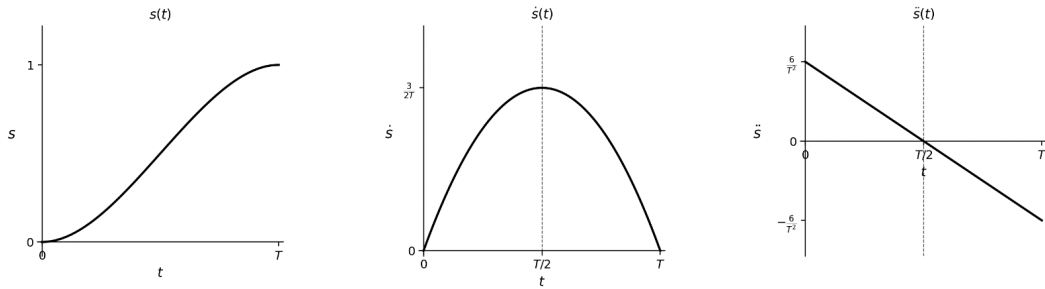


Figure 4.3: Cubic polynomial profile: zero velocity at endpoints with non-zero endpoint accelerations.

4.2.5 Quintic Polynomial Profile

The quintic profile is governed by six boundary conditions:

$$s(0) = 0, \quad s(T) = 1, \quad \dot{s}(0) = \dot{s}(T) = 0, \quad \ddot{s}(0) = \ddot{s}(T) = 0, \quad (4.14)$$

which uniquely determine the three non-zero polynomial coefficients:

$$s(t) = a_3t^3 + a_4t^4 + a_5t^5, \quad (4.15)$$

$$a_3 = \frac{10}{T^3}, \quad a_4 = -\frac{15}{T^4}, \quad a_5 = \frac{6}{T^5}. \quad (4.16)$$

Peak velocity at $t = T/2$:

$$\dot{s}_{\max} = \frac{15}{8T}. \quad (4.17)$$

The minimum traversal time for a joint segment with limit $\dot{\theta}_{\max,i}$ is:

$$T \geq \frac{15}{8} \max_i \frac{|\Delta\theta_i|}{\dot{\theta}_{\max,i}}. \quad (4.18)$$

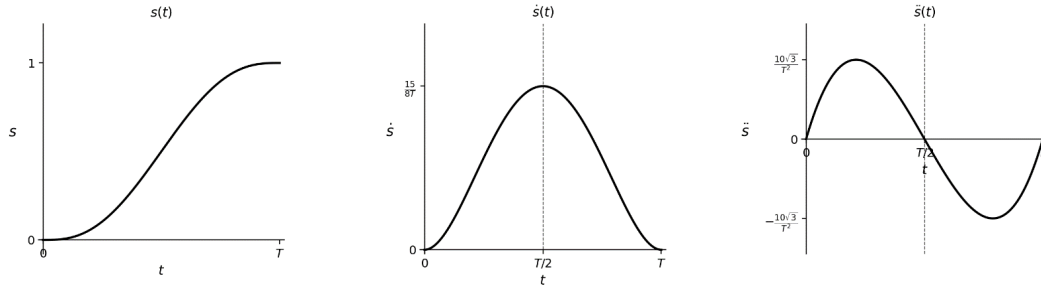


Figure 4.4: Quintic polynomial profile: zero velocity and zero acceleration at both endpoints.

4.2.6 Time-Optimal Path Parametrisation (TOPP-RA)

Given a geometric path $\mathbf{q}(s)$, TOPP-RA finds the time parametrisation $s(t)$ minimising traversal time subject to all joint constraints. Differentiating $\mathbf{q}(s)$ twice and substituting into Eq. (3.11) transforms the joint-space constraints into scalar inequalities on $(\dot{s}, \ddot{s})$:

$$\mathbf{a}(s)\ddot{s} + \mathbf{b}(s)\dot{s}^2 + \mathbf{c}(s) \in \mathcal{C}(s), \quad (4.19)$$

where $\mathcal{C}(s)$ is a convex polytope encoding torque and velocity bounds. The algorithm solves two passes of N small linear programmes, achieving $\mathcal{O}(mN)$ complexity.

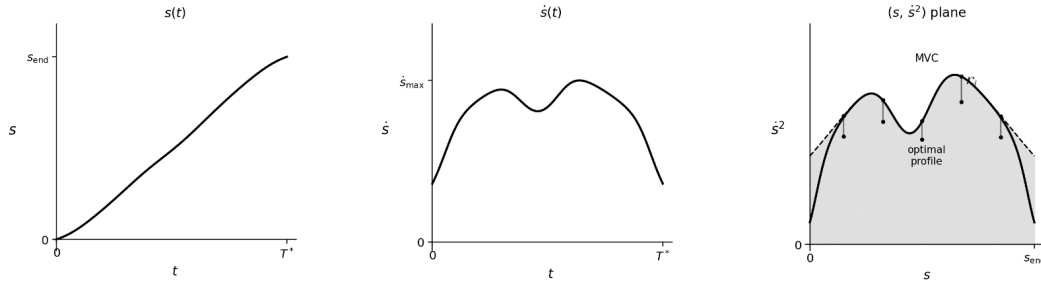


Figure 4.5: TOPP-RA time-optimal velocity profile, saturating both velocity and torque limits simultaneously.

4.2.7 Profile Comparison

Table 4.1: Comparison of the five trajectory profiles on motion quality, constraint enforcement, and computational requirements.

Criterion	Trapezoidal	S-Curve	Cubic	Quintic	TOPP-RA
Vel. continuity	C^0 (jumps in $\dot{s}$)	C^1	C^1 at endpoints	C^2	C^2 path-dependent
Acceleration	Discontinuous	Continuous, bounded jerk	Non-zero at endpoints	Zero at endpoints	Continuous
Jerk	Infinite at transitions	Bounded J	Infinite at $t=0, T$	Zero at endpoints	Not bounded
Traversal time	Min. for given v, a	$\approx 3\text{-}5\%$ longer	User-prescribed	Longer than cubic	Provably optimal
Peak $\dot{s}_{\max}$	v	v	$3/(2T)$	$15/(8T)$	Saturates $\dot{\theta}_{\max}$
Torque limits	Not enforced	Not enforced	Not enforced	Not enforced	Hard LP constraint
Complexity	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(mN)$
Vibration	High	Low	High at endpoints	Low	Moderate
Grasping suitability	Limited	Good	Limited	Good	Best

C^n : n -th derivative continuous. All complexity figures for single-segment single-DoF.

Chapter 5

Manipulator Control

5.1 Feedback Linearisation (Computed Torque)

5.1.1 Motivation

Classical linear controllers such as PID are only locally valid near a linearisation point. For a manipulator, the full nonlinear dynamics—inertial coupling, Coriolis effects, and gravity—change substantially across the workspace. Feedback linearisation resolves this by exactly cancelling all nonlinearities through the control input, transforming the closed-loop system into decoupled linear integrators valid throughout the operating region.

5.1.2 Control Law

With output $\mathbf{y} = \boldsymbol{\theta}$ the relative degree is $r = 2$. The input transformation:

$$\boldsymbol{\tau} = \mathbf{M}(\boldsymbol{\theta})\mathbf{v} + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}} + \mathbf{g}(\boldsymbol{\theta}), \quad (5.1)$$

reduces the closed-loop to $\ddot{\boldsymbol{\theta}} = \mathbf{v}$. A PD law with feedforward acceleration:

$$\mathbf{v} = \ddot{\boldsymbol{\theta}}_d + \mathbf{K}_d(\dot{\boldsymbol{\theta}}_d - \dot{\boldsymbol{\theta}}) + \mathbf{K}_p(\boldsymbol{\theta}_d - \boldsymbol{\theta}), \quad (5.2)$$

gives the complete computed-torque law:

$$\boldsymbol{\tau} = \mathbf{M}(\boldsymbol{\theta})\left[\ddot{\boldsymbol{\theta}}_d + \mathbf{K}_d(\dot{\boldsymbol{\theta}}_d - \dot{\boldsymbol{\theta}}) + \mathbf{K}_p(\boldsymbol{\theta}_d - \boldsymbol{\theta})\right] + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}} + \mathbf{g}(\boldsymbol{\theta}). \quad (5.3)$$

5.1.3 Tracking Error Dynamics and Stability

Defining $\mathbf{e} = \boldsymbol{\theta}_d - \boldsymbol{\theta}$, the closed-loop error equation is:

$$\ddot{\mathbf{e}} + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e} = \mathbf{0}. \quad (5.4)$$

The natural frequency and damping ratio per joint are:

$$\omega_n = \sqrt{K_p}, \quad \zeta = \frac{K_d}{2\omega_n} = \frac{K_d}{2\sqrt{K_p}}. \quad (5.5)$$

5.1.4 Lyapunov Stability Analysis

Consider the candidate Lyapunov function:

$$V = \frac{1}{2} \dot{\boldsymbol{\theta}}^\top \mathbf{M}(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}} + \frac{1}{2} \mathbf{e}^\top \mathbf{K}_p \mathbf{e} > 0. \quad (5.6)$$

Differentiating and using the skew-symmetry of $(\dot{\mathbf{M}} - 2\mathbf{C})$:

$$\dot{V} = -\dot{\boldsymbol{\theta}}^\top \mathbf{K}_d \dot{\boldsymbol{\theta}} \leq 0. \quad (5.7)$$

By LaSalle's invariance principle, the equilibrium $(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) = (\boldsymbol{\theta}_d, \mathbf{0})$ is globally asymptotically stable.

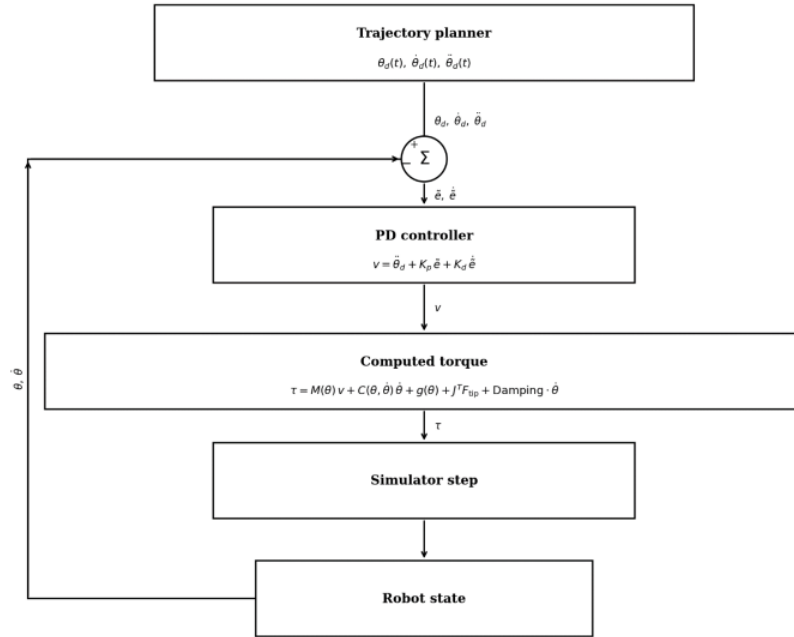


Figure 5.1: Block diagram of the computed-torque (feedback linearisation) controller. The nonlinear cancellation term $\mathbf{M}\mathbf{v} + \mathbf{C}\dot{\boldsymbol{\theta}} + \mathbf{g}$ transforms the plant into decoupled double integrators driven by the PD auxiliary input.

5.2 Adaptive Sliding Mode Control

5.2.1 Motivation

The computed-torque law assumes exact knowledge of $\mathbf{M}$, $\mathbf{C}$, $\mathbf{g}$. In practice, inertia parameters are only approximately known and external disturbances are unavoidable. Sliding Mode Control (SMC) exploits the manipulator's structural dynamics to guarantee trajectory tracking despite bounded modelling errors, without requiring precise knowledge of uncertainty bounds.

5.2.2 Sliding Surface

Define the tracking error $\mathbf{e} = \boldsymbol{\theta}_d - \boldsymbol{\theta}$. The sliding surface vector is:

$$\mathbf{s} = \dot{\mathbf{e}} + \boldsymbol{\Lambda}\mathbf{e} = \dot{\boldsymbol{\theta}} - \dot{\boldsymbol{\theta}}_r, \quad \dot{\boldsymbol{\theta}}_r = \dot{\boldsymbol{\theta}}_d - \boldsymbol{\Lambda}\mathbf{e}, \quad (5.8)$$

where $\boldsymbol{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_7) \succ 0$.

5.2.3 Control Law

$$\boldsymbol{\tau} = \hat{\mathbf{M}}(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}}_r + \hat{\mathbf{C}}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}}_r + \hat{\mathbf{g}}(\boldsymbol{\theta}) - \mathbf{k} \text{sgn}(\mathbf{s}), \quad (5.9)$$

where $\ddot{\boldsymbol{\theta}}_r = \ddot{\boldsymbol{\theta}}_d - \boldsymbol{\Lambda}\dot{\mathbf{e}}$ and switching gains k_i satisfy $k_i \geq \|\hat{\mathbf{M}}\ddot{\boldsymbol{\theta}}_r + \tilde{\mathbf{C}}\dot{\boldsymbol{\theta}}_r + \tilde{\mathbf{g}}\|_i + \eta_i$ (hat: estimated model; tilde: model error).

5.2.4 Chattering Reduction via Boundary Layer

The sign function is replaced by a saturation function within boundary layer $\Phi_i > 0$:

$$\text{sat}(s_i/\Phi_i) = \begin{cases} s_i/\Phi_i & |s_i| < \Phi_i \\ \text{sgn}(s_i) & \text{otherwise.} \end{cases} \quad (5.10)$$

5.2.5 Adaptive Switching Gains

$$k_i(t) = k_{\min,i} + \alpha_{\text{adapt},i} |s_i(t)|, \quad (5.11)$$

$$k_i(t) \leftarrow \text{clip}(k_i(t), k_{\min,i}, k_{\max,i}). \quad (5.12)$$

5.2.6 Lyapunov Stability Analysis

With $V = \frac{1}{2} \mathbf{s}^\top \mathbf{M}(\boldsymbol{\theta}) \mathbf{s}$, applying the skew-symmetry of $(\dot{\mathbf{M}} - 2\mathbf{C})$ and the gain condition yields:

$$\dot{V} \leq - \sum_{i=1}^7 \eta_i |s_i| \leq 0. \quad (5.13)$$

The sliding surface $\mathbf{s} = \mathbf{0}$ is reached in finite time:

$$t_r \leq \frac{\sqrt{2V(0)/\alpha_M}}{\min_i(\eta_i)}, \quad (5.14)$$

where α_M is the minimum eigenvalue of $\mathbf{M}$.

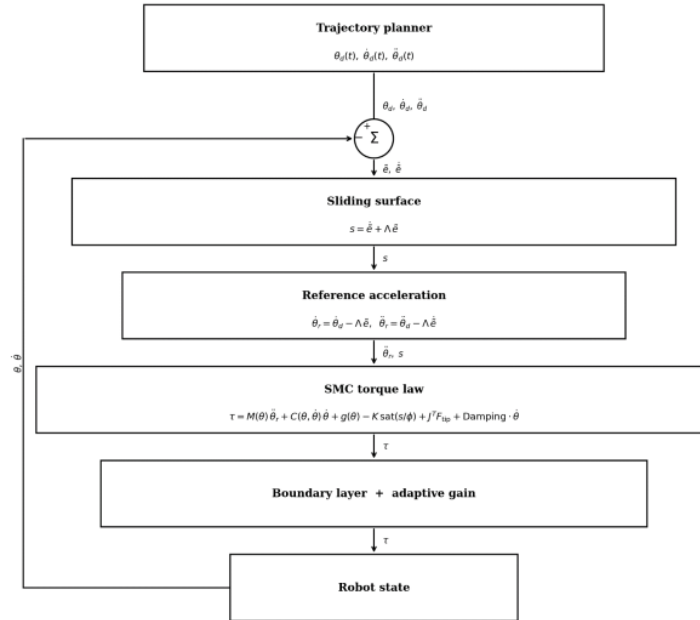


Figure 5.2: Block diagram of the adaptive sliding mode controller. The switching term $\mathbf{k}\text{sat}(\mathbf{s}/\Phi)$ drives the state onto the sliding surface $\mathbf{s} = \mathbf{0}$; the adaptive gain law prevents unnecessary chattering.

5.3 Nonlinear Model Predictive Control

5.3.1 Receding-Horizon Principle and Advantages

Model Predictive Control (MPC) is an optimisation-based control strategy that computes control actions by repeatedly solving a finite-horizon Optimal Control Problem (OCP) at each time step. The controller uses a dynamic model of the system to predict future behaviour over a prediction horizon of N steps, then selects the control sequence that

minimises a cost function over that window subject to system and actuator constraints. Only the first control input of the optimal sequence is applied; the horizon then shifts forward by one step and the process repeats—the *receding horizon principle*.

The key advantages of MPC for robot manipulation are:

- **Constraint handling:** joint limits, torque bounds, and velocity limits are enforced as hard constraints directly in the optimisation, rather than through ad-hoc saturations.
- **Anticipation:** the controller sees N steps ahead, allowing it to slow down before a joint limit or pre-accelerate along a trajectory.
- **Systematic tuning:** performance trade-offs (tracking accuracy vs. control effort) are encoded explicitly in the cost function weights $\mathbf{Q}$ and $\mathbf{R}$.

5.3.2 Optimal Control Problem Formulation

At each MPC step, the following finite-horizon OCP is solved:

$$\min_{\substack{\mathbf{x}_0, \dots, \mathbf{x}_N \\ \mathbf{u}_0, \dots, \mathbf{u}_{N-1}}} \sum_{k=0}^{N-1} [\mathbf{e}_k^\top \mathbf{Q} \mathbf{e}_k + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k] + \mathbf{e}_N^\top \mathbf{Q}_f \mathbf{e}_N, \quad (5.15)$$

subject to:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \quad (5.16)$$

$$\mathbf{x}_0 = \mathbf{x}_{\text{current}}, \quad (5.17)$$

$$\mathbf{q}_{\min} \leq \mathbf{q}_k \leq \mathbf{q}_{\max}, \quad \dot{\mathbf{q}}_{\min} \leq \dot{\mathbf{q}}_k \leq \dot{\mathbf{q}}_{\max}, \quad (5.18)$$

$$\boldsymbol{\tau}_{\min} \leq \mathbf{u}_k \leq \boldsymbol{\tau}_{\max}, \quad (5.19)$$

where:

- $\mathbf{x}_k = [\mathbf{q}_k^\top, \dot{\mathbf{q}}_k^\top]^\top \in \mathbb{R}^{14}$: system state (joint positions and velocities).
- $\mathbf{u}_k \in \mathbb{R}^7$: control input (joint torques).
- $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}_k^{\text{ref}}$: tracking error relative to the reference trajectory.
- $\mathbf{Q} \succeq 0, \mathbf{R} \succ 0$: state and control weighting matrices.
- $\mathbf{Q}_f \succeq 0$: terminal cost weight.
- $f(\cdot)$: discrete-time robot dynamics.

5.3.3 Robot Dynamics and Discretisation

The robot dynamics follow the standard rigid-body equation of motion:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}, \quad (5.20)$$

where $\mathbf{M}(\mathbf{q})$ is the joint-space inertia matrix, $\mathbf{C}\dot{\mathbf{q}}$ collects Coriolis and centrifugal forces, and $\mathbf{g}(\mathbf{q})$ is the gravitational torque vector.

Solving for joint acceleration and discretising with step size h :

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1} [\boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q})], \quad (5.21)$$

$$\mathbf{x}_{k+1} = \begin{bmatrix} \mathbf{q}_k + h\dot{\mathbf{q}}_k \\ \dot{\mathbf{q}}_k + h\ddot{\mathbf{q}}_k \end{bmatrix}. \quad (5.22)$$

All rigid-body quantities ($\mathbf{M}$, $\mathbf{C}\dot{\mathbf{q}}$, $\mathbf{g}$) are computed symbolically using Pinocchio-CasADi: the CRBA provides $\mathbf{M}(\mathbf{q})$ and the RNEA provides the bias force $\mathbf{b}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{C}\dot{\mathbf{q}} + \mathbf{g}$. Because both return CasADi symbolic expressions, the entire dynamics function is fully differentiable—exact first- and second-order derivatives flow automatically into the NLP solver.

5.3.4 Multiple-Shooting Transcription

To solve the OCP numerically, it is transcribed into a finite-dimensional Nonlinear Program (NLP) using *multiple shooting*: both the state trajectory $\mathbf{X} = \{\mathbf{x}_0, \dots, \mathbf{x}_N\}$ and the control sequence $\mathbf{U} = \{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$ are declared as independent decision variables, and the dynamics are imposed as equality constraints:

$$\mathbf{x}_{k+1} - f(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{0}, \quad k = 0, \dots, N-1. \quad (5.23)$$

This yields a larger but sparser NLP, whose block-tridiagonal Jacobian structure is fully exploited by the solver, making multiple shooting significantly faster and more numerically stable than single shooting for the horizon lengths used here ($N \in [5, 40]$).

5.3.5 CasADi/IPOPT Implementation

The NLP is assembled once symbolically in CasADi and solved at each MPC step by IPOPT (Interior Point OPTimiser), which applies a primal-dual interior-point method. CasADi provides exact sparse Jacobians and Hessians via algorithmic differentiation, allowing IPOPT to exploit the exact second-order structure of the problem.

Warm starting: because consecutive MPC problems share nearly identical structure, the solution from step k provides an excellent initial guess for step $k+1$. After applying $\mathbf{u}_0^*$, the state and control trajectories are shifted forward by one stage and the Lagrange multipliers are reused as a dual warm start. In practice, this reduces solve time by 3–5 $\times$ compared to cold starting.

Although the CasADi/IPOPT implementation was developed and validated first, profiling revealed per-step solve times of 50–200 ms—far exceeding the 5 ms MPC step

required for dynamic grasping. It was therefore replaced by the ACADOS framework for all experimental evaluations.

5.3.6 ACADOS SQP-RTI Implementation

ACADOS replaces IPOPT with a tailored real-time optimal control framework using the Sequential Quadratic Programming Real-Time Iteration (SQP-RTI) scheme: exactly one linearisation and one QP solve per control step, making the per-step computation time nearly constant and predictable.

Continuous-time dynamics are integrated internally using an explicit Runge-Kutta scheme:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}^{-1}(\mathbf{q})[\boldsymbol{\tau} - \mathbf{b}(\mathbf{q}, \dot{\mathbf{q}})] \end{bmatrix}. \quad (5.24)$$

The QP at each RTI step is solved by HPIPM (High-Performance Interior-Point Method), which exploits the block-diagonal Hessian structure via partial condensing. The solver is generated offline as compiled C code; the online loop only calls the pre-compiled solver functions, adding negligible overhead. Compared to the CasADi-IPOPT approach, ACADOS achieves substantially lower per-step solve times, making it suitable for high-frequency control of the Franka arm.

5.4 Gripper Mechanics and Contact Grasping

5.4.1 Force Closure and Friction Constraints

A grasp achieves *force closure* if the contact forces can resist any external wrench. The Coulomb no-slip condition at each pad is:

$$F_i \leq \mu_{\text{eff}} N_i, \quad \mu_{\text{eff}} = \frac{\mu_{\text{sphere}} + \mu_{\text{finger}}}{2}. \quad (5.25)$$

The minimum normal force per finger to resist gravity is:

$$F_{\min} = \frac{mg}{2\mu_{\text{eff}}}. \quad (5.26)$$

5.4.2 Two-Phase Gripper Closure

Trajectory phase: The finger aperture decreases linearly:

$$q_g(t) = 0.04 - \rho(t) d_{\text{closure}}, \quad d_{\text{closure}} = 0.04 - r_{\text{sphere}}. \quad (5.27)$$

Force-regulated phase: A compressive squeeze force $F_g = -\alpha_g F_{\min}$ ($\alpha_g > 1$) is

applied until $f_{\text{contact}} > F_{\text{min}}$.

5.4.3 Grasp Matrix Analysis

For a parallel-jaw grasp on a sphere, the grasp matrix $\mathbf{G} \in \mathbb{R}^{6 \times 2}$ has only the F_x row non-zero under frictionless contact, giving $\text{rank}(\mathbf{G}) = 1$. With Coulomb friction the polyhedral cone approximation achieves $\text{rank}(\mathbf{G}) = 6$ (force closure) once the normal force exceeds F_{min} .

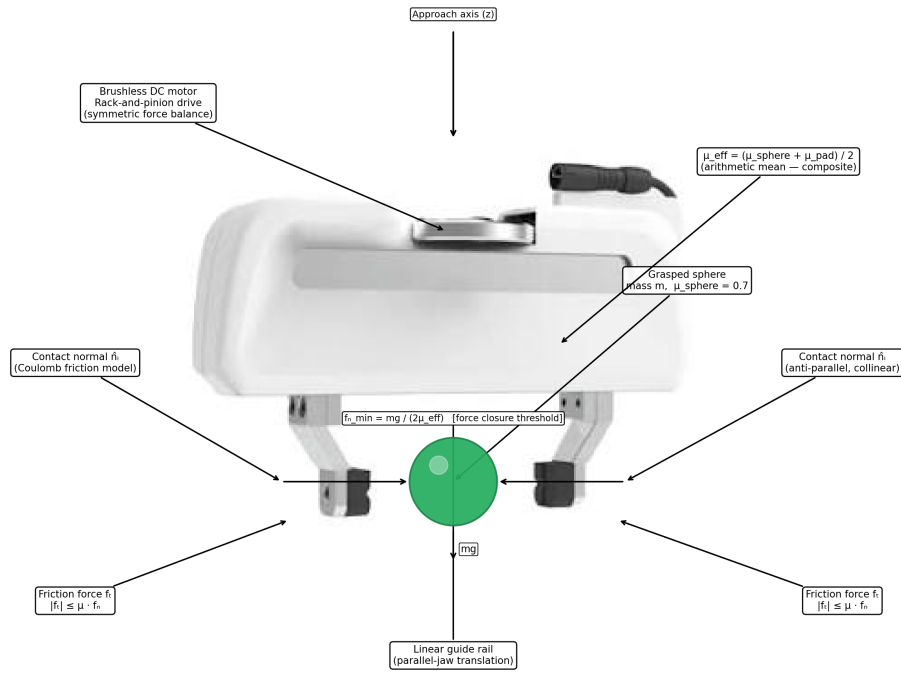


Figure 5.3: Franka Hand parallel-jaw gripper and contact geometry on the spherical object.

Chapter 6

Experimental Results and Dynamic Grasping

6.1 IK Solver Evaluation

6.1.1 Experimental Setup

A set of $N = 500$ reachable target poses was generated by sampling joint configurations uniformly from $[\boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}]$ (RNG seed 0) and computing end-effector poses via Pinocchio forward kinematics. The initial joint configuration $\mathbf{q}_0$ was fixed to the zero vector (home configuration) for all trials, providing a consistent and realistic starting point independent of the target pose. Two solvers were evaluated: (i) an *analytical* solver (**frantik**) sweeping θ_7 over a discrete set of random seeds; (ii) a *numerical* Pinocchio-DLS solver with $\lambda^2 = 10^{-4}$ and a maximum of 25 iterations per seed. All random seeds were drawn uniformly from $[\mathbf{Q}_{\min}, \mathbf{Q}_{\max}]$ with no use of $\mathbf{q}_0$ as a seed, and solutions were scored by proximity to the home configuration.

6.1.2 Results

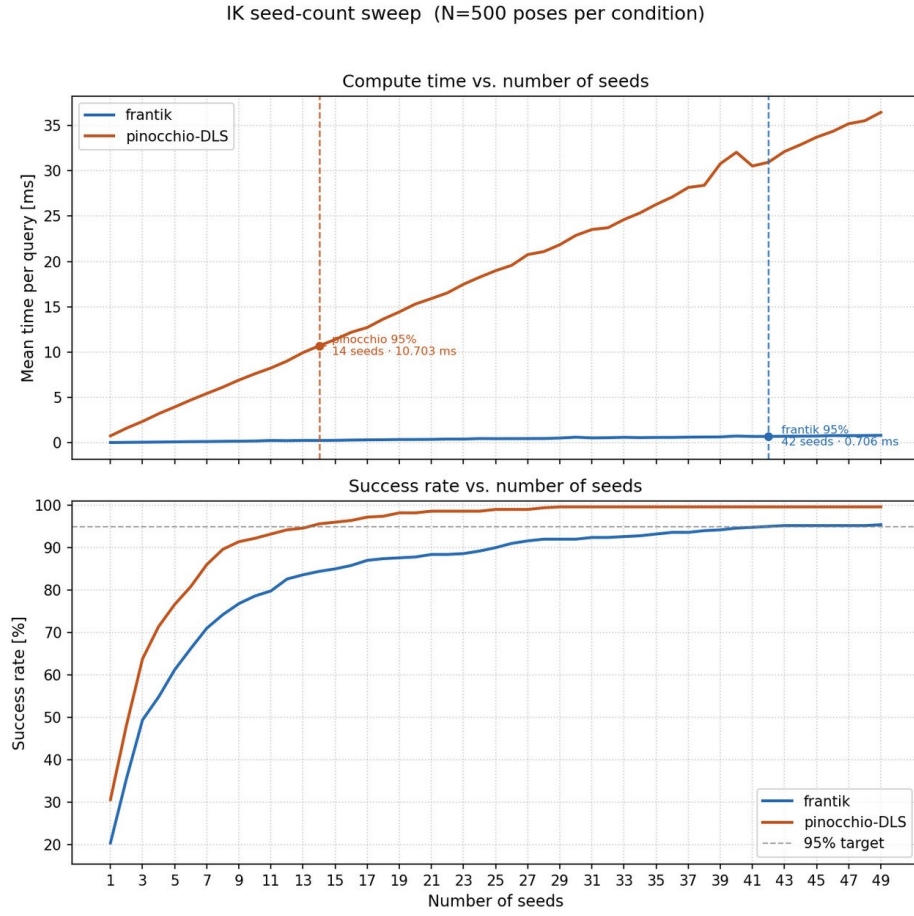


Figure 6.1: IK seed-count sweep ($N = 500$ poses per condition). *Upper:* mean solve time per query (ms). *Lower:* success rate (%). Blue: analytical solver (**frantik**); Orange: Pinocchio-DLS numerical solver. Dashed vertical lines mark the 95% success threshold for each solver.

Table 6.1: Operating-point metrics for both IK solvers at the 95% success threshold.

Metric	Analytical (frantik)	Numerical (Pinocchio-DLS)
Seeds to reach 95% success	42	14
Mean time at 95% threshold	0.706 ms	10.703 ms
Time per seed	~ 0.017 ms	~ 0.76 ms
Speed-up at 95% threshold	15 $\times$ faster (analytical)	
Success ceiling ($n = 49$)	$\sim 95.4\%$	$\sim 99.6\%$

The results reveal a clear trade-off between the two solvers. The analytical solver (**frantik**) reaches 95% success at 42 seeds in only 0.706 ms, owing to its extremely low per-seed cost of approximately 0.017 ms—each seed involves a single closed-form

evaluation with no iteration. The numerical solver (Pinocchio-DLS) reaches the same 95% threshold with far fewer seeds (14), but at a significantly higher cost of 10.703 ms, as each seed requires up to 25 Newton–Raphson iterations (~ 0.76 ms/seed).

A key limitation of the analytical solver is its structural ceiling: success rate plateaus near 95–97% regardless of seed count, due to singular configurations where the closed-form solution is analytically undefined (specifically, configurations where joint 2 approaches zero). The numerical solver does not share this ceiling and continues improving with more seeds, reaching 99.6% at $n = 49$.

Given the requirement for repeated IK calls in the intercept planner at real-time rates, the analytical solver with $n = 42$ seeds is the preferred choice, delivering 95% success in under 1 ms. Where maximum reachability is critical and latency is less constrained, the numerical solver with 14+ seeds provides a higher success ceiling.

6.2 Feedback Linearisation Experiments

6.2.1 Point-Stabilisation Damping-Ratio Study

The computed-torque controller was tested on a joint-space point-stabilisation task with three damping ratios ($\zeta = 0.5, 1.0, 1.5$). The robot moved from the all-zero home configuration to $\mathbf{q}^* = [2.433, 0.028, -2.424, -2.729, 0.044, 2.707, 0.754]$ rad.

The per-joint natural frequencies used are $\omega_n = [16, 8, 12, 12, 24, 24, 32]$ rad/s, giving $k_{p,j} = \omega_{n,j}^2$ and $k_{d,j} = 2\zeta \omega_{n,j}$ for each joint. Table 6.2 lists the resulting gains for all three cases.

Table 6.2: Per-joint CT gains for the three damping-ratio cases ($k_p = \omega_n^2$, $k_d = 2\zeta\omega_n$).

Joint	ω_n (rad/s)	k_p	k_d ($\zeta=0.5$)	k_d ($\zeta=1.0$)	k_d ($\zeta=1.5$)
J1	16	256	16	32	48
J2	8	64	8	16	24
J3	12	144	12	24	36
J4	12	144	12	24	36
J5	24	576	24	48	72
J6	24	576	24	48	72
J7	32	1024	32	64	96

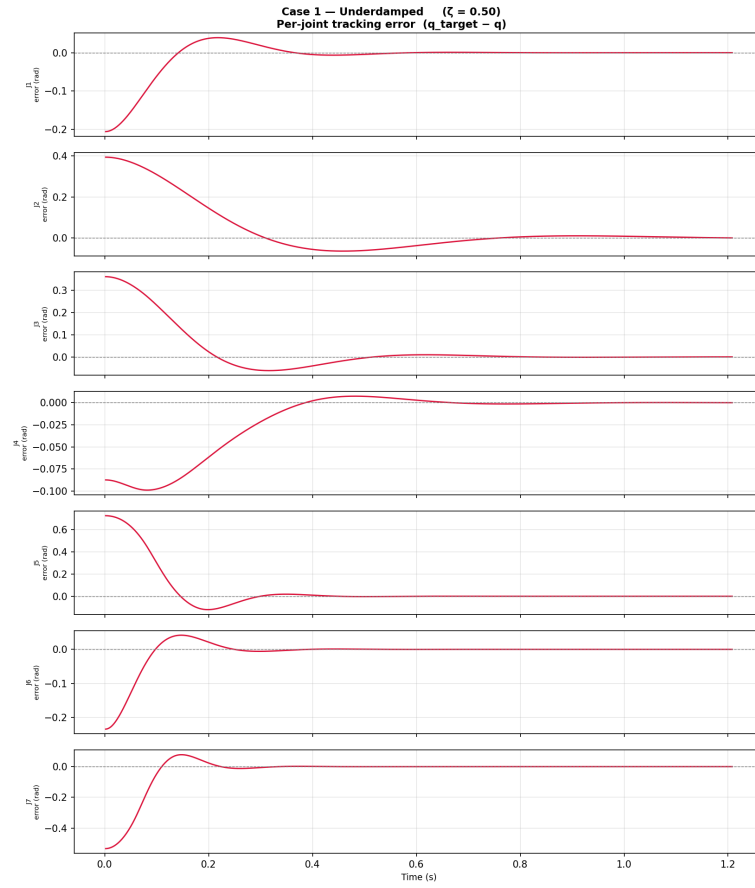


Figure 6.2: Computed-torque stabilisation, underdamped ($\zeta = 0.5$): per-joint tracking error.

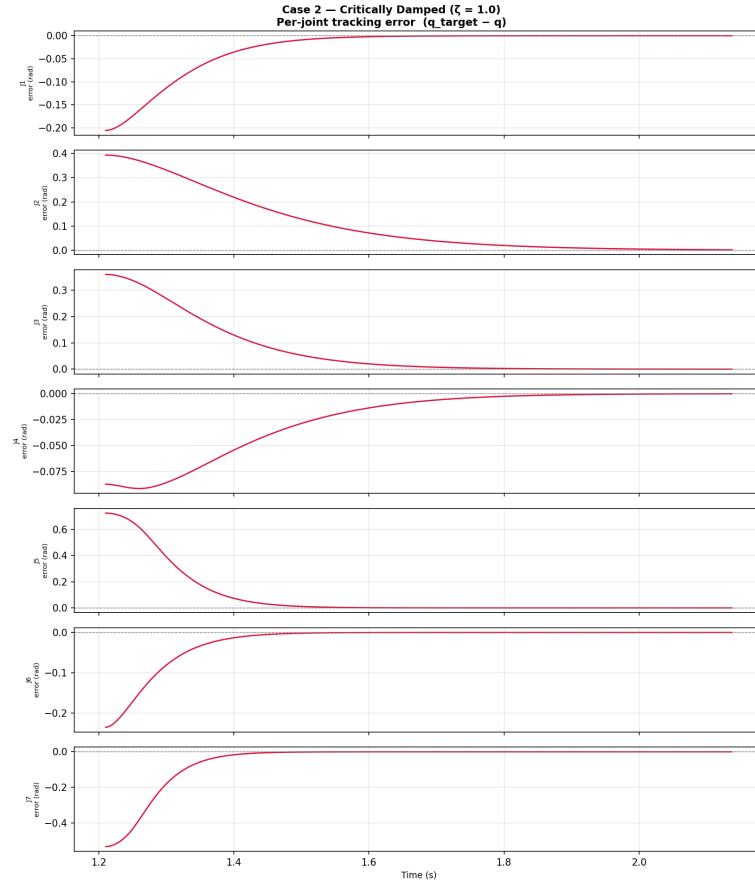


Figure 6.3: Computed-torque stabilisation, critically damped ($\zeta = 1.0$): per-joint tracking error.

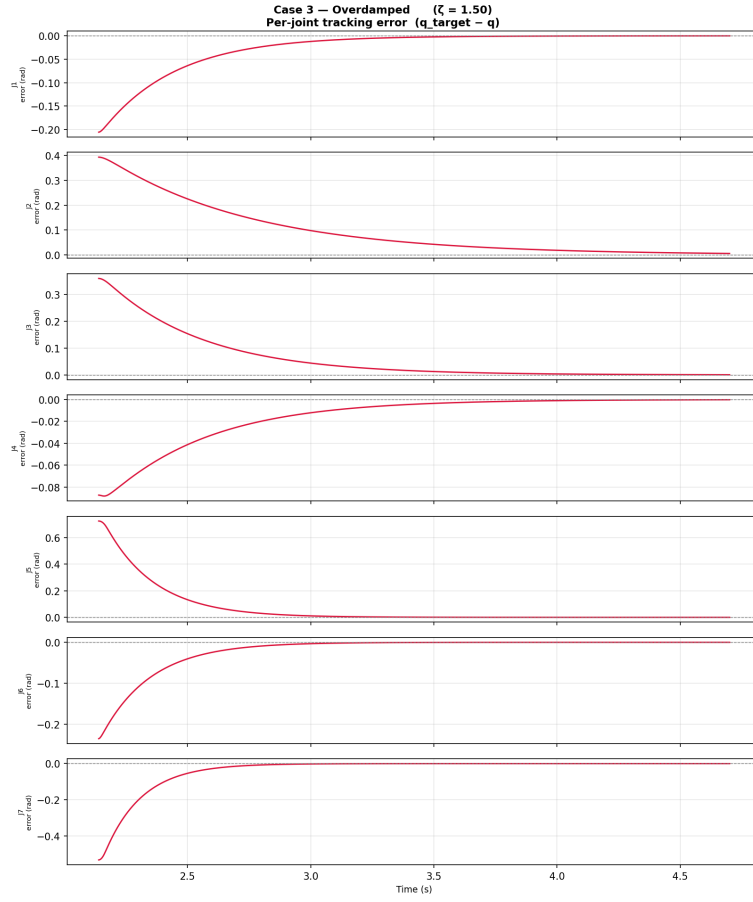


Figure 6.4: Computed-torque stabilisation, overdamped ($\zeta = 1.5$): per-joint tracking error.

Table 6.3: Joint-averaged performance comparison across three damping cases.

Metric	Case 1 ($\zeta = 0.5$)	Case 2 ($\zeta = 1.0$)	Case 3 ($\zeta = 1.5$)
Mean overshoot (%)	15.69	0.00	0.00
Mean rise time (s)	0.12	0.26	0.76
Mean settling time (s)	0.55	1.67	3.63
Mean ITAE ($\times 10^{-3}$)	3.18	65.7	282.9
Mean τ_{rms} (N·m)	9.15	7.91	5.94

Practical selection: For the approach phase, $\zeta = 0.5$ provides fastest settling (0.55 s) with acceptable overshoot. For the final grasp-hold, $\zeta = 1.0$ eliminates overshoot and collision risk. Overdamped $\zeta = 1.5$ is not suitable for any phase of a dynamic grasping task due to its slow settling time and higher ITAE compared to $\zeta = 0.5$.

6.2.2 Trajectory Tracking

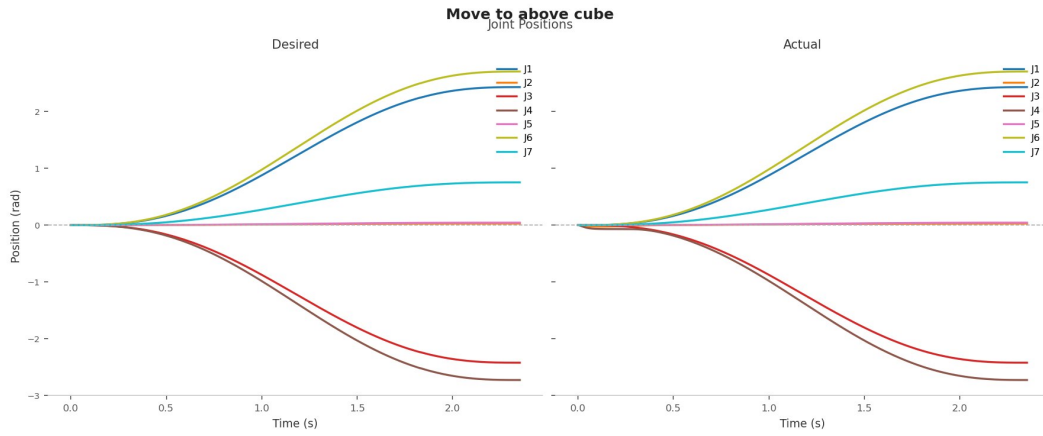


Figure 6.5: Feedback linearisation trajectory tracking with quintic profile: joint-space error.

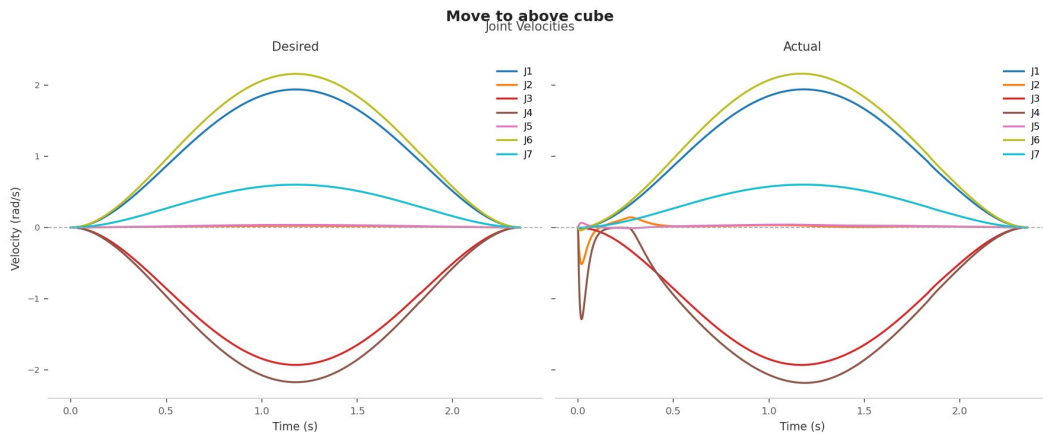


Figure 6.6: Feedback linearisation trajectory tracking with quintic profile: velocity profiles.

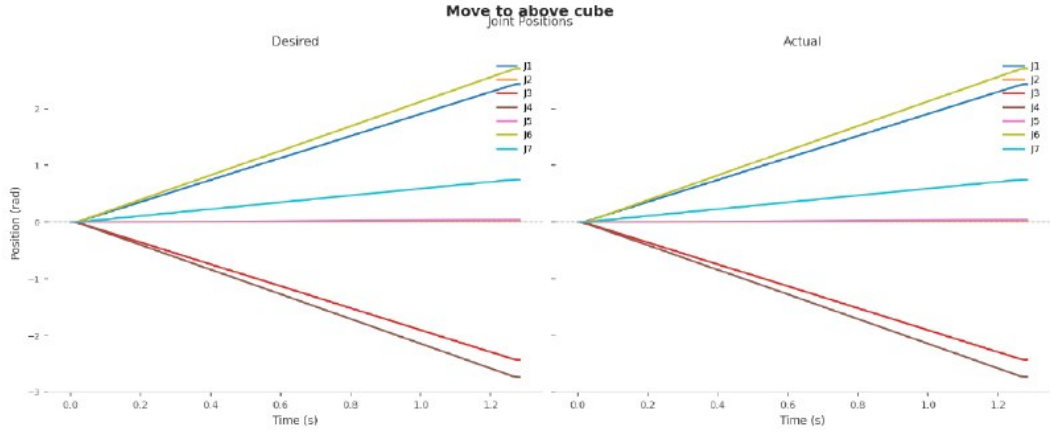


Figure 6.7: Feedback linearisation trajectory tracking with TOPP-RA profile: joint position actual vs desired.

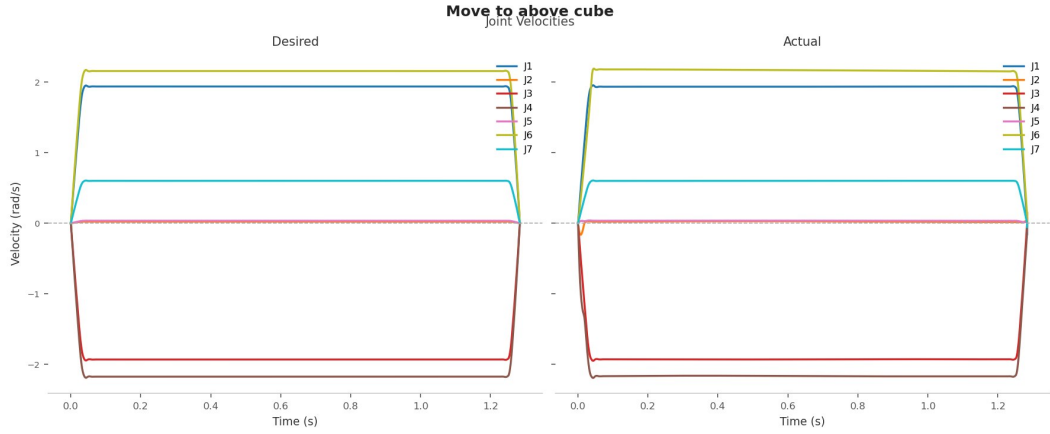


Figure 6.8: Feedback linearisation trajectory tracking with TOPP-RA profile: tracking error.

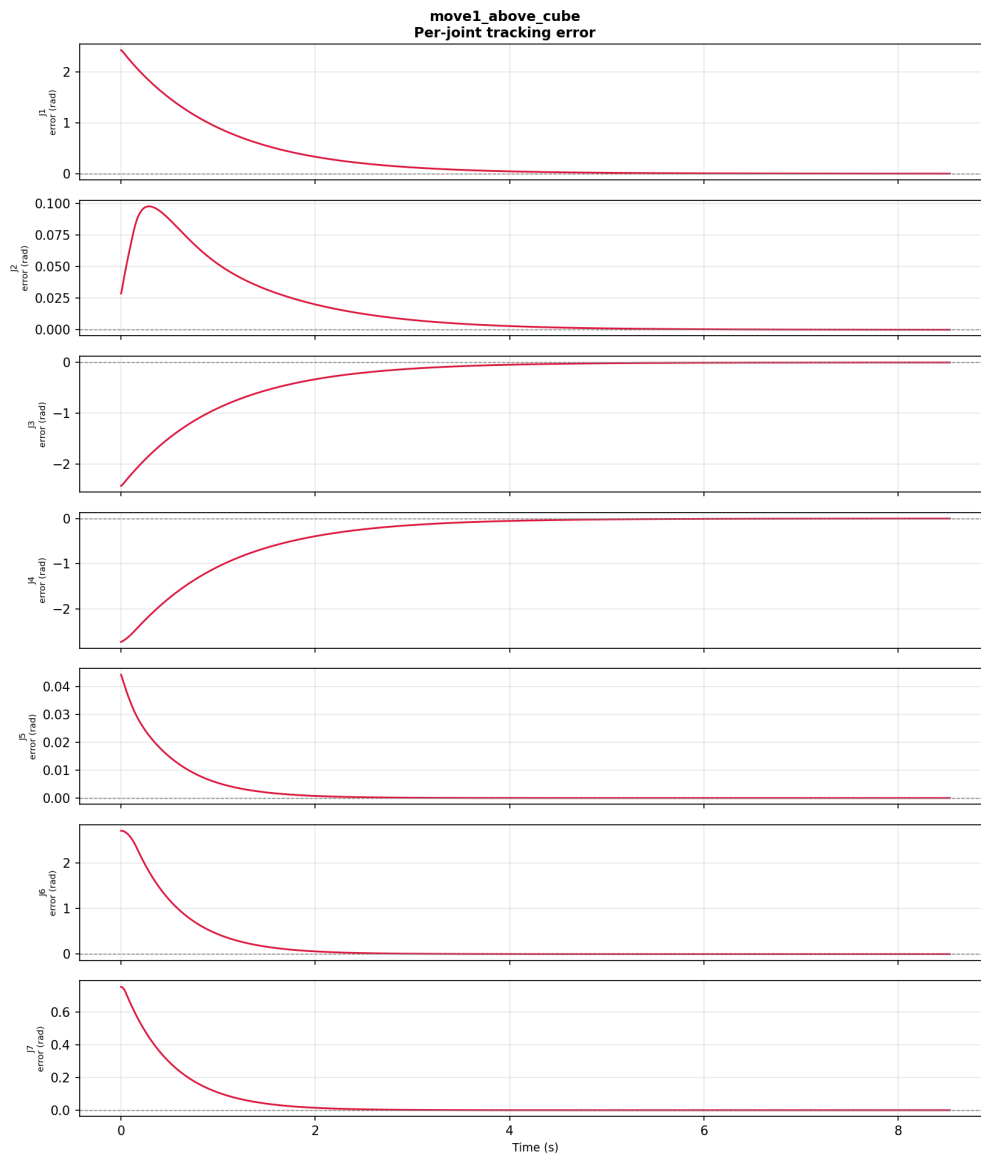
6.3 Sliding Mode Control Experiments

6.3.1 Point Stabilisation

The robot moved from $\mathbf{q}_{\text{start}} = \mathbf{0}$ to the pre-grasp target $\mathbf{q}^*$, the same large-displacement configuration used in the FL experiment. Termination criterion: $\max_j |e_j(t)| < 1 \text{ mrad}$.

Table 6.4: Per-joint SMC metrics over the full regulation manoeuvre (8.534s total).

Joint	λ_j	K_j	φ_j	ISE	IAE	ITAE	e_{rms}	s_{max}	τ_{rms}
J1	1	20	0.5	3.004	2.453	2.454	0.594	2.433	0.851
J2	1	20	0.5	0.007	0.128	0.142	0.029	0.493	14.57
J3	1	15	0.4	2.974	2.439	2.439	0.592	2.424	0.865
J4	1	10	0.3	4.103	2.878	2.901	0.695	2.729	17.58
J5	2	5	0.2	0.000	0.020	0.010	0.007	0.089	0.378
J6	2	5	0.2	2.434	1.581	0.805	0.536	5.414	2.553
J7	2	3	0.2	0.156	0.396	0.199	0.136	1.508	0.224

**Figure 6.9:** SMC per-joint tracking error $e_j(t)$: all joints converge monotonically to below 1 mrad, confirming boundary-layer saturation is active throughout.

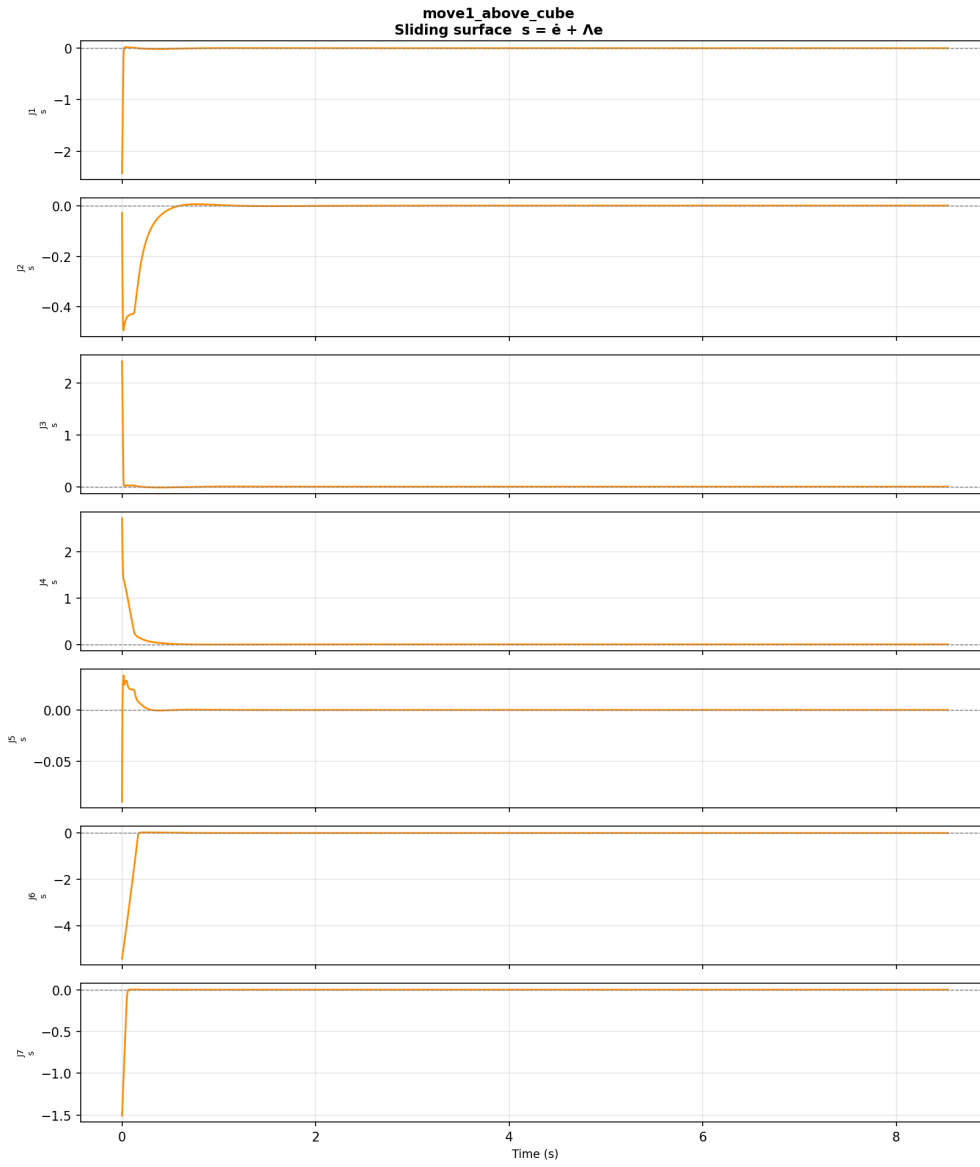


Figure 6.10: SMC sliding surface $s_j(t) = \dot{e}_j + \lambda_j e_j$: zero high-frequency oscillation confirms the saturation function suppresses chattering.

6.3.2 Trajectory Tracking with SMC

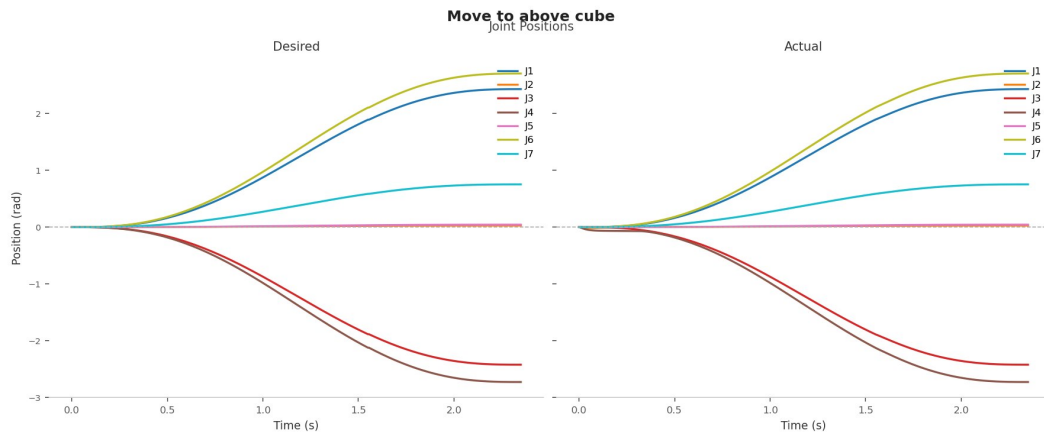


Figure 6.11: SMC trajectory tracking with quintic profile: per-joint error.

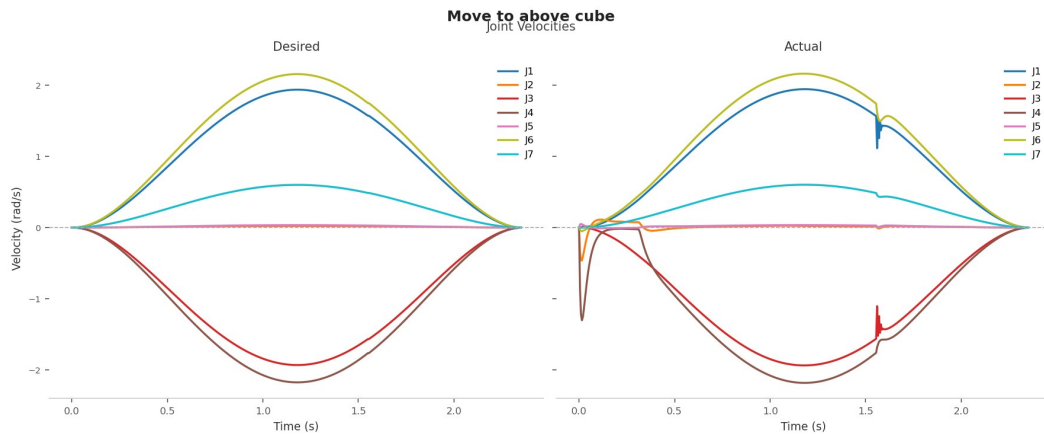


Figure 6.12: SMC trajectory tracking with quintic profile: actual vs desired velocity.

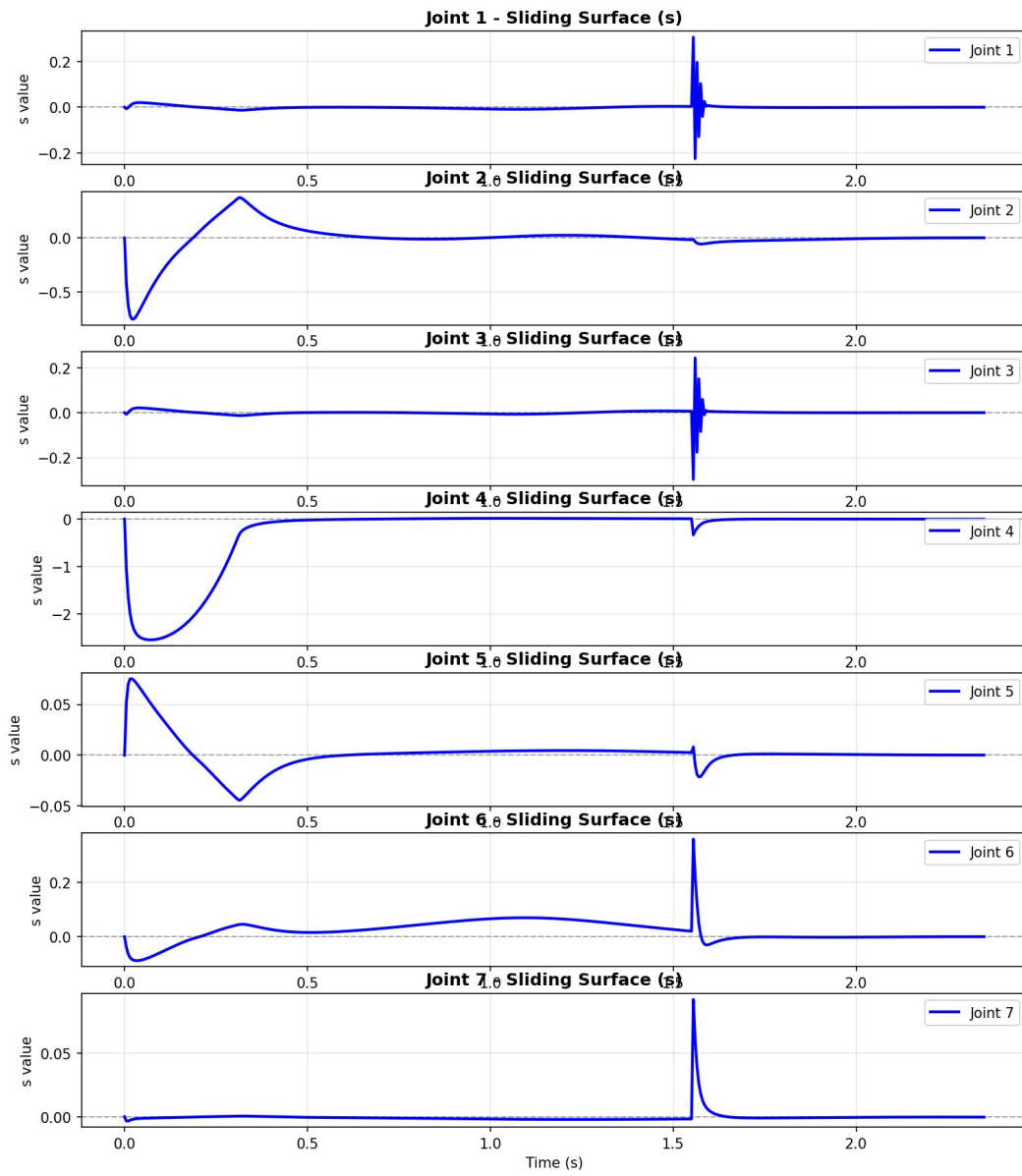
Sliding Surface (s) for All 7 Joints

Figure 6.13: SMC trajectory tracking with quintic profile: slide surface values.

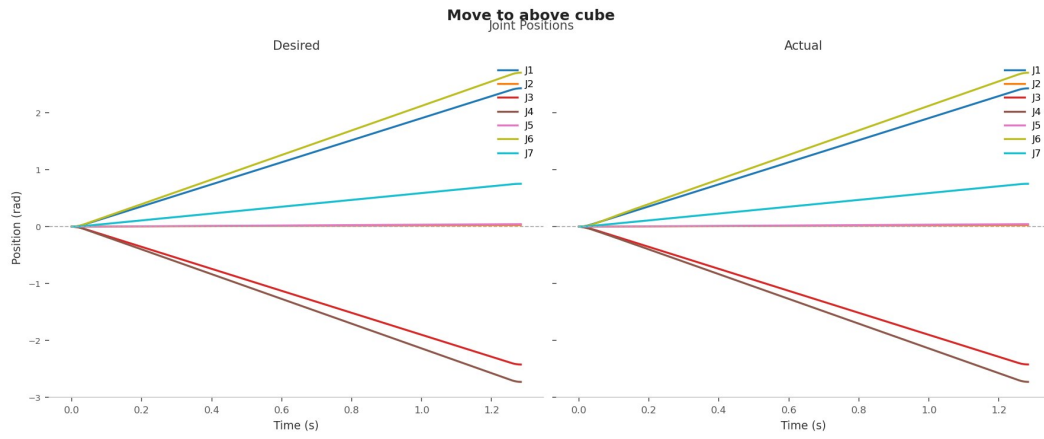


Figure 6.14: SMC trajectory tracking with TOPP-RA profile: joint position actual vs desired.

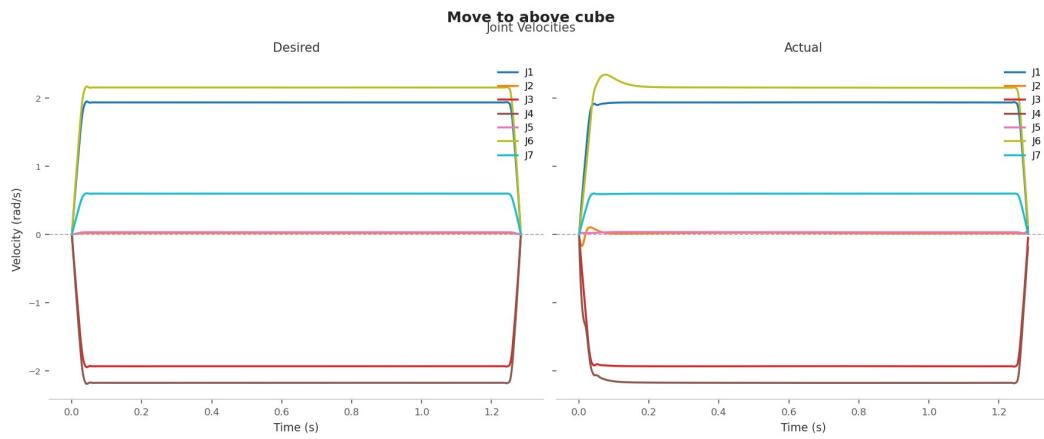


Figure 6.15: SMC trajectory tracking with TOPP-RA profile: joint velocities actual vs desired.

Sliding Surface (s) for All 7 Joints

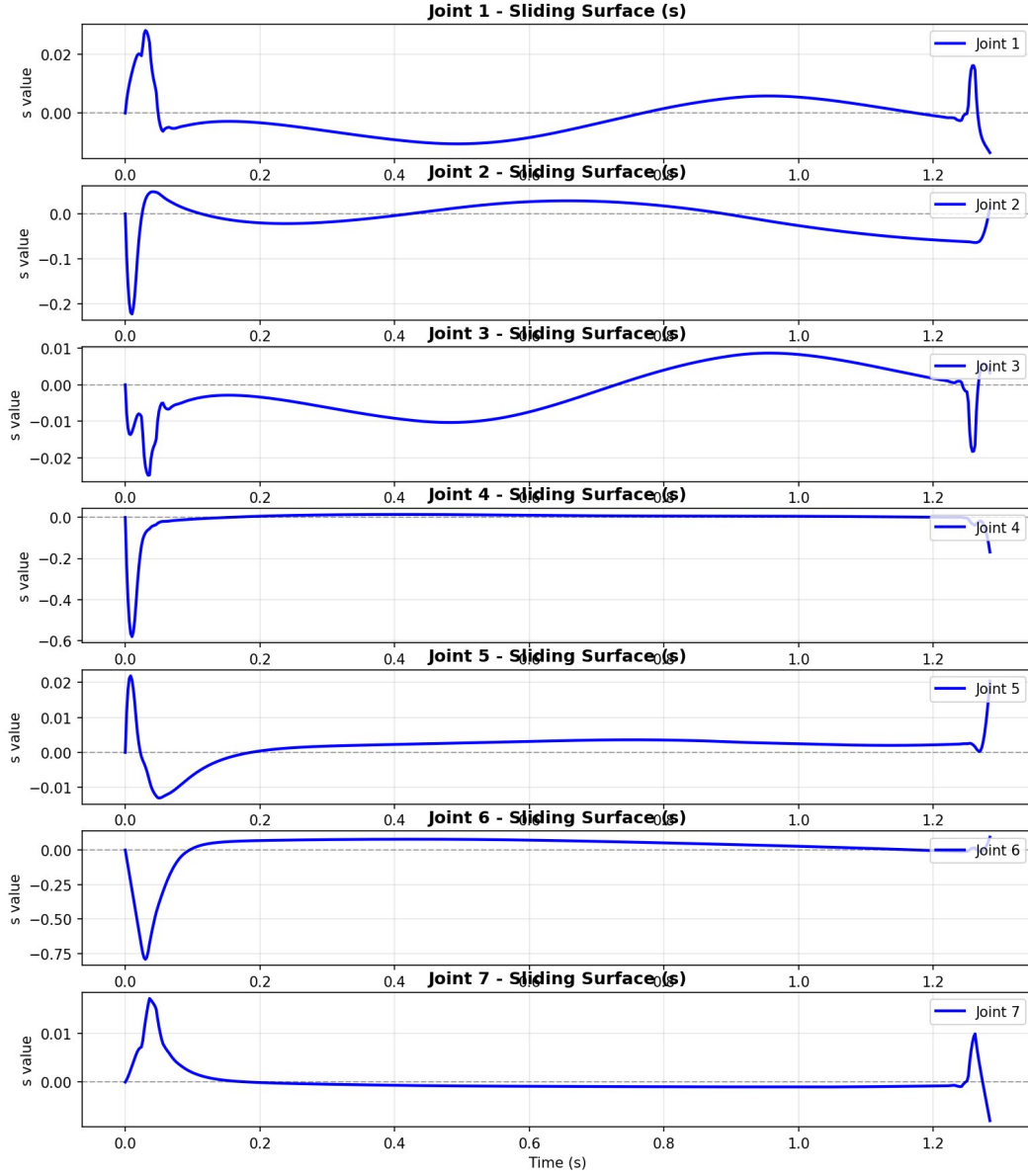


Figure 6.16: SMC trajectory tracking with TOPP-RA profile: sliding surface values of the joints.

6.4 MPC Point Stabilisation

6.4.1 Setup and Results

The ACADOS SQP-RTI controller drove the Panda from a near-zero initial configuration ($q_4 = -0.1$ rad, all others zero) to the pre-grasp target $\mathbf{q}^*$ used in the FL and SMC experiments.

The following MPC parameters were used:

- Step size: $h = 0.01$ s
- Prediction horizon: $N = 10$ steps
- State cost: $\mathbf{Q} = 200 \cdot \text{diag}(300\mathbf{I}_7, 30\mathbf{I}_7)$
- Control cost: $\mathbf{R} = \text{diag}(0.5\mathbf{I}_7)$

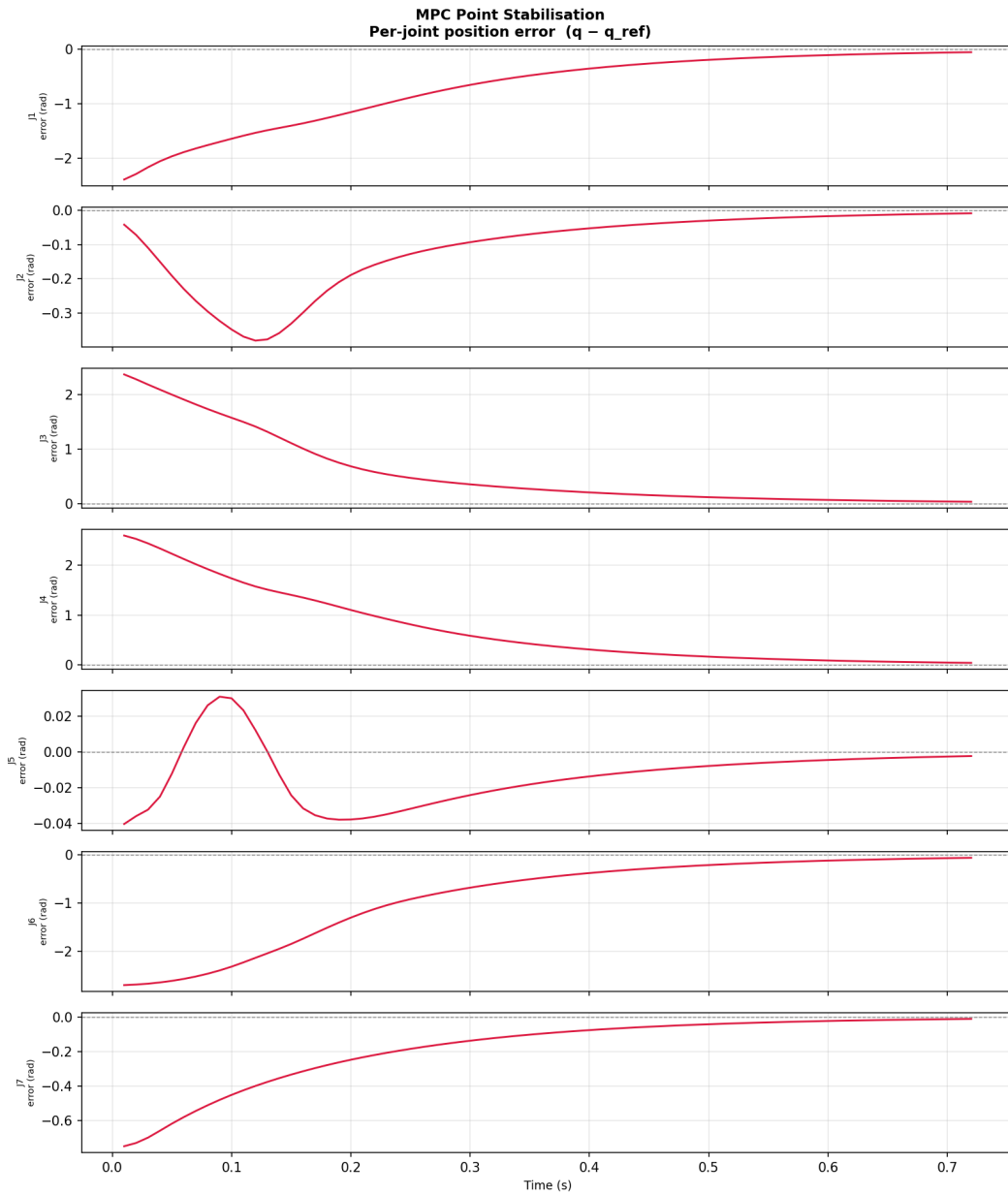


Figure 6.17: MPC point stabilisation: per-joint position error $e_j(t) = q_j(t) - q_j^*$.

Table 6.5: Per-joint MPC stabilisation performance metrics.

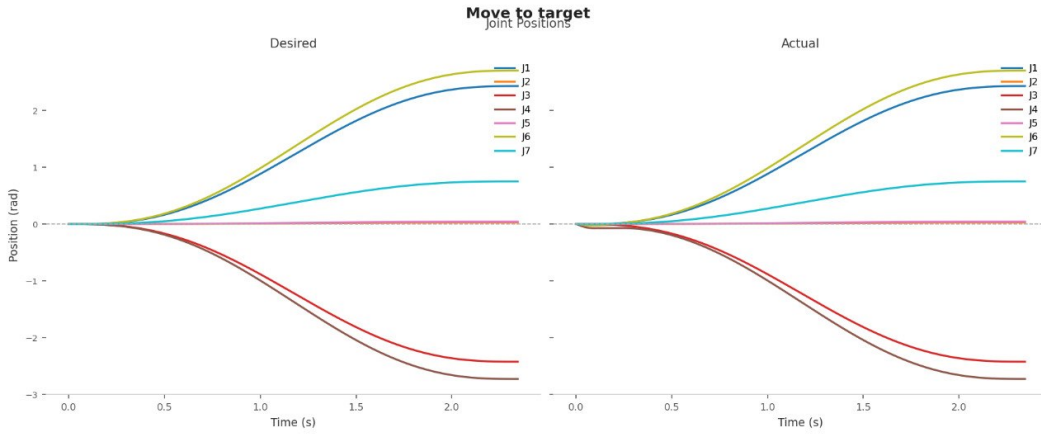
Joint	ISE	IAE	ITAE	e_{rms}	t_s (s)	τ_{rms} (N·m)
J1	0.636	0.492	0.094	0.981	0.720	30.066
J2	0.017	0.077	0.015	0.154	0.720	45.716
J3	0.494	0.384	0.062	0.874	0.660	36.613
J4	0.700	0.494	0.088	1.032	0.680	38.778
J5	0.000	0.012	0.003	0.020	0.720	5.612
J6	1.032	0.600	0.108	1.239	0.720	6.406
J7	0.048	0.124	0.021	0.272	0.670	2.799

Mean ACADOS solve time was 25.16 ms per step.

6.4.2 Trajectory Tracking with MPC

The following MPC parameters were used for trajectory tracking:

- Step size: $h = 0.005$ s
- Prediction horizon: $N = 8$ steps
- State cost: $\mathbf{Q} = 200 \cdot \text{diag}(300\mathbf{I}_7, 30\mathbf{I}_7)$
- Control cost: $\mathbf{R} = \text{diag}(\mathbf{I}_7)$

**Figure 6.18:** MPC trajectory tracking with quintic profile.

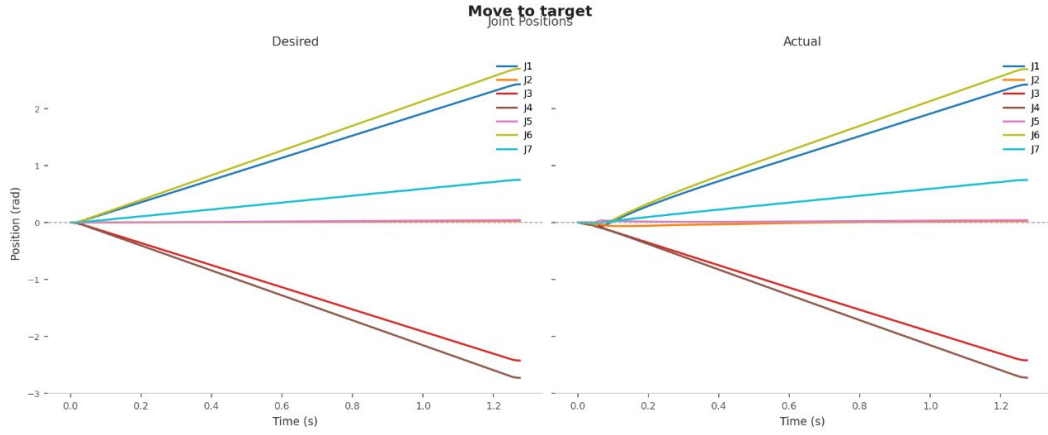


Figure 6.19: MPC trajectory tracking with TOPP-RA profile.

6.5 Gripper Closure Experiment

The Franka Hand gripper was tested in a two-phase closure sequence. In the first (trajectory) phase, the fingers are commanded to close from the full-open aperture (0.08 m) to just above the sphere diameter using a position-controlled trajectory. In the second (force-regulation) phase, the gripper applies a controlled squeeze torque until the contact normal force reaches the minimum threshold $F_{\min}$ required for force closure (Eq. 5.26), then holds at that force level.

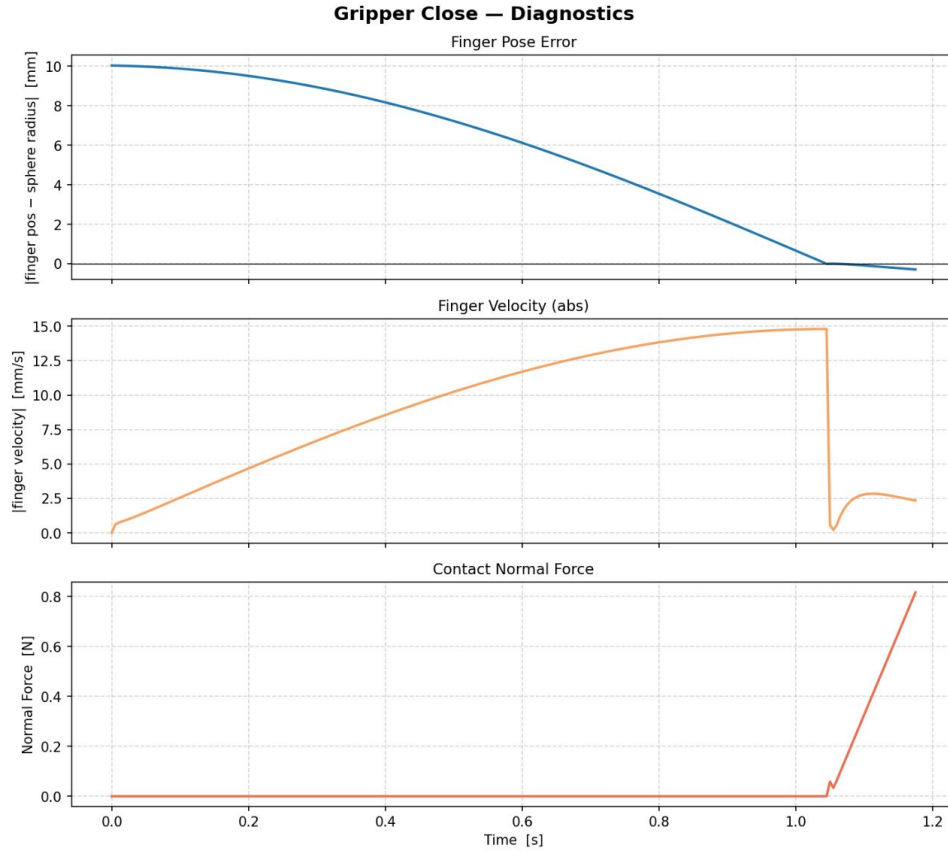


Figure 6.20: Gripper closure experiment.

During Phase 1, the proportional controller drives the finger toward the sphere, so the pose error decreases smoothly while velocity builds up as the finger accelerates. At contact (≈ 1.05 s), the finger arrives with ≈ 15 mm/s velocity, which causes the small normal-force spike and the sudden velocity drop as the sphere acts as a mechanical stop. The slightly negative pose error after contact is due to MuJoCo’s compliant contact model allowing a small penetration of the finger past the sphere surface. From that point, Phase 2 takes over and the normal force of one of the fingers of the gripper rises steadily until grip is achieved.

6.6 Controller Benchmark

Three controllers were benchmarked on $N = 300$ target configurations paired with two trajectory planners (TOPPRA and Quintic), under three conditions: nominal, torque disturbance ($\sigma = 2$ N·m, 80 simulation timesteps), and sensor noise ($\sigma = 0.002$ rad).

Table 6.6: Mean tracking RMSE (rad) over 300 seeds, three conditions.

Condition	MPC·T	MPC·Q	SMC·T	SMC·Q	FL·T	FL·Q
Nominal	0.00694	0.00737	0.01413	0.01783	0.01843	0.02118
Disturbance ($\sigma=2\text{ N}\cdot\text{m}$)	0.00722	0.00763	0.01430	0.01798	0.01922	0.02200
Sensor noise	0.00694	0.00737	0.01421	0.01790	0.01847	0.02117

T: TOPPRA; Q: Quintic; FL: Feedback Linearisation.

Table 6.7: Control effort, computation cost, and joint constraint violations.

Metric	MPC·T	MPC·Q	SMC·T	SMC·Q	FL·T	FL·Q
Total energy ($\text{N}^2\text{m}^2\text{s}$)	1778.1	1985.6	1995.2	1944.6	2034.3	2073.5
Mean jerk norm	1.061	0.106	1.180	0.332	1.050	0.161
Step time (ms)	3.686	3.411	0.142	0.125	0.069	0.061
Joint violation (%)	0.00	0.00	0.00	0.23	0.21	0.69
Settling time (s)	1.125	1.846	1.131	1.854	1.132	1.854

The benchmark results reveal several key observations. **Tracking accuracy:** MPC achieves the lowest mean RMSE across all three conditions (0.00694 rad nominal with TOPPRA), outperforming SMC by approximately $2\times$ and FL by approximately $2.7\times$. The performance advantage is consistent under both disturbance injection and sensor noise, confirming that the model-based prediction gives MPC a structural advantage over the reactive controllers.

Control energy: MPC also achieves the lowest total torque energy ($1778\text{ N}^2\text{m}^2\text{s}$ with TOPPRA), despite its more complex optimisation, because the OCP explicitly penalises $\mathbf{u}^\top \mathbf{R} \mathbf{u}$ at every step. SMC’s adaptive switching gain causes it to inject additional high-frequency energy, particularly in the disturbance case.

Disturbance rejection: SMC exhibits the best disturbance rejection in relative terms: its RMSE increases by only 1.2% under torque disturbance ($0.01413 \rightarrow 0.01430$ rad with TOPPRA), compared to 4.1% for MPC and 4.3% for FL. The inherent robustness of sliding mode control to bounded matched disturbances is confirmed.

Computation cost: FL evaluates in < 0.07 ms per step, SMC in 0.12–0.14 ms, and MPC in 3.4–3.7 ms. SMC and FL are therefore $25\text{--}55\times$ faster than MPC.

Trajectory planner effect: Settling times cluster by trajectory planner rather than by controller (TOPPRA ≈ 1.13 s, Quintic ≈ 1.85 s across all three controllers), confirming that the choice of reference generator has as much impact on closed-loop performance as the control law itself.

Joint constraint violations: MPC respects all joint limits (0.00% violation) due to

explicit constraint enforcement in the OCP. SMC and FL exhibit small violations (up to 0.69% for FL with Quintic), arising from overshoot in joints with large displacements.

Based on these results, **MPC with TOPP-RA** (MPC·T) is selected as the controller for the dynamic grasping experiments in the subsequent sections. It delivers the best tracking accuracy and lowest control energy, and the 3.7 ms per-step solve time is comfortably within the 5 ms MPC step used in the dynamic grasping loop.

6.7 Static Grasping

Before addressing dynamic interception, the static grasping scenario validates the complete pipeline. The robot executes four phases: (1) approach to a pre-grasp pose above the sphere, (2) descent to the grasp pose, (3) two-phase gripper closure (Section 5.4), and (4) lift and transport.

To validate the complete manipulation pipeline prior to dynamic experiments, the static grasping scenario was evaluated across multiple ball positions distributed throughout the arm’s reachable workspace. In each trial, the robot was required to approach a stationary sphere, descend to the grasp pose, execute the two-phase gripper closure sequence, and subsequently lift and transport the object to an alternative location. Across all tested configurations, the arm successfully intercepted the sphere and relocated it, confirming that the integrated kinematic, dynamic, and gripper subsystems operate correctly end-to-end and that the system is ready to proceed to dynamic grasping experiments.

6.8 Dynamic Grasping Algorithms

6.8.1 Intercept-Point Planning

The intercept planner performs a forward scan over candidate rendezvous times t . At each candidate, workspace feasibility is checked, IK is solved, minimum quintic segment durations are estimated, and the feasibility criterion $T_1 + T_2 + T_{\text{grip}} + \varepsilon \leq t$ is evaluated.

Algorithm 3 Feasible Intercept Search (Forward Scan)**Require:** Ball state $(\mathbf{p}_b, \mathbf{v}_b)$, arm state $(\mathbf{q}, \dot{\mathbf{q}})$ **Ensure:** Intercept time t^* , configuration $\mathbf{q}^*$ (or None)

```

1:  $t \leftarrow t_{\min}; \Delta t \leftarrow 0.05 \text{ s}$ 
2: while  $t \leq t_{\max}$  do
3:    $\mathbf{p}_{\text{int}} \leftarrow \mathbf{p}_b + \mathbf{v}_b \cdot t$  ▷ linear prediction
4:   if  $\mathbf{p}_{\text{int}} \notin \mathcal{W}$  then
5:      $t \leftarrow t + \Delta t$ ; continue
6:   end if
7:    $\mathbf{T}_g \leftarrow \text{GRASPPOSE}(\mathbf{p}_{\text{int}}, \mathbf{v}_b)$ 
8:    $(\mathbf{q}^*, \text{ok}) \leftarrow \text{IK}(\mathbf{T}_g, \mathbf{q})$ 
9:   if not ok then
10:     $t \leftarrow t + \Delta t$ ; continue
11:  end if
12:   $\mathbf{T}_{\text{hover}} \leftarrow \text{PREGRASPPOSE}(\mathbf{p}_{\text{int}}, +0.10 \text{ m})$ 
13:   $(\mathbf{q}_{\text{hover}}, \text{ok}) \leftarrow \text{IK}(\mathbf{T}_{\text{hover}}, \mathbf{q})$ 
14:  if not ok then
15:     $t \leftarrow t + \Delta t$ ; continue
16:  end if
17:   $T_1 \leftarrow \text{QuinticDuration}(\mathbf{q} \rightarrow \mathbf{q}_{\text{hover}})$ 
18:   $T_2 \leftarrow \text{QuinticDuration}(\mathbf{q}_{\text{hover}} \rightarrow \mathbf{q}^*)$ 
19:  if  $T_1 + T_2 + T_{\text{grip}} + \varepsilon \leq t$  then
20:    return  $(t, \mathbf{q}^*)$ 
21:  end if
22:   $t \leftarrow t + \Delta t$ 
23: end while
24: return None

```

6.8.2 Accelerated Intercept Search (CROFT)

CROFT casts the intercept search as a scalar root-finding problem. Define the signed feasibility function:

$$f(t) = T_{\text{arm}}(t) + T_{\text{grip}} - t, \quad (6.1)$$

where $T_{\text{arm}}(t)$ is the minimum time the arm requires to reach the predicted ball position at candidate intercept time t (derived from IK and a quintic trajectory duration estimate), and T_{grip} is the gripper closure time. A negative $f(t)$ means the arm arrives with time to spare (feasible); a positive $f(t)$ means the arm is too slow (infeasible). The earliest feasible intercept t^* is the root of $f(t) = 0$.

Phase 1 — Coarse bracket: The algorithm scans candidate times from $t_{\min}$ to $t_{\max}$ in steps of $\Delta t_c = 0.3 \text{ s}$. At each step it checks workspace feasibility of the predicted ball position, solves IK for the corresponding grasp pose, and evaluates $f(t)$. The scan stops as soon as a sign change is detected, yielding a bracket $[t_{\text{low}}, t_{\text{high}}]$ that straddles

the root.

Phase 2 — Hybrid secant/bisection (Illinois): The bracket is refined over up to 10 iterations. Each iteration attempts a secant step

$$t_{\text{new}} = t_{\text{high}} - f_{\text{high}} \frac{t_{\text{high}} - t_{\text{low}}}{f_{\text{high}} - f_{\text{low}}}; \quad (6.2)$$

if the secant point falls outside $[t_{\text{low}}, t_{\text{high}}]$, a bisection midpoint is used instead. An Illinois correction (halving the stale function value) is applied when the same endpoint is updated on two successive iterations, preventing superlinear degradation. Iterations stop when $|f_{\text{high}}| < 0.05$ s (the feasibility residual is small enough) or when $|t_{\text{high}} - t_{\text{low}}| < 10^{-4}$ s (the bracket has collapsed).

Phase 3 — TOPP-RA refinement: Once the root is bracketed tightly, TOPP-RA is applied once to the converged intercept candidate to enforce joint torque and velocity limits, producing a time-optimal, constraint-satisfying trajectory. The final margin $t^* - T_{\text{toppra}} > 0$ confirms that the arm can execute the trajectory before the ball arrives.

Algorithm 4: CROFT Accelerated Intercept Search

Require: Ball position $\mathbf{p}_b$, velocity $\mathbf{v}_b$, arm config $\mathbf{q}_0$

Ensure: Intercept result $\{t^*, T_1, T_2, T_g, \mathbf{q}_{pg}, \mathbf{q}_g, \text{trajectories}\}$ or None

```

1: function FEASIBILITY( $t_c$ )
2:    $\mathbf{p}_{\text{int}} \leftarrow \mathbf{p}_b + \mathbf{v}_b t_c$ 
3:   if  $\mathbf{p}_{\text{int}} \notin \mathcal{W}$  or IK for grasp/pregrasp fails then return (None, None)
4:   end if
5:    $T_1 \leftarrow \text{QUINTICDUR}(\mathbf{q}_0 \rightarrow \mathbf{q}_{pg}); T_2 \leftarrow \text{QUINTICDUR}(\mathbf{q}_{pg} \rightarrow \mathbf{q}_g)$ 
6:   return ( $T_1 + T_2 + T_{\text{grip}} - t_c$ , payload)  $\triangleright f \leq 0$ : feasible;  $f > 0$ : infeasible
7: end function

Phase 1 — Coarse bracket scan:
8:  $t_{\text{low}} \leftarrow \text{None}; t_{\text{high}} \leftarrow \text{None}$ 
9: for  $t = t_{\text{min}}$  to  $t_{\text{max}}$  step  $\Delta t_c$  do
10:  ( $f$ , payload)  $\leftarrow$  FEASIBILITY( $t$ ); if  $f = \text{None}$ : continue
11:  if  $f > 0$  then  $t_{\text{low}} \leftarrow t; f_{\text{low}} \leftarrow f$ 
12:  else  $t_{\text{high}} \leftarrow t; f_{\text{high}} \leftarrow f; \text{bp} \leftarrow \text{payload};$  break
13:  end if
14: end for
15: if  $t_{\text{high}} = \text{None}$  then return None
16: end if
17:  $t^* \leftarrow t_{\text{high}}$ 

```

```

18: if  $t_{\text{low}} \neq \text{None}$  then  $\triangleright$  bracket  $[t_{\text{low}}, t_{\text{high}}]$  found; refine with Illinois secant/bisection
    Phase 2 — Illinois hybrid secant/bisection:
19:   for  $i = 1, \dots, N_{\text{iter}}$  do
20:     if  $|f_{\text{high}}| < \varepsilon$  or  $|t_{\text{high}} - t_{\text{low}}| < 10^{-4}$  then break
21:     end if
22:     Try secant step; fall back to bisection midpoint if secant leaves bracket
23:      $(f_{\text{new}}, \text{payload}_{\text{new}}) \leftarrow \text{FEASIBILITY}(t_{\text{new}})$ 
24:     if  $f_{\text{new}} > 0$  then update  $t_{\text{low}}$ ; apply Illinois correction if low side stagnates
25:     else  $t_{\text{high}} \leftarrow t_{\text{new}}$ ;  $t^* \leftarrow t_{\text{new}}$ ;  $\text{bp} \leftarrow \text{payload}_{\text{new}}$ 
26:     end if
27:   end for
28: end if
    Phase 3 — TOPP-RA refinement:
29:  $(T_1, T_2, T_g, \text{traj}_1, \text{traj}_2) \leftarrow \text{TOPPRA}(\text{bp}.\mathbf{q}_0, \text{bp}.\mathbf{q}_{pg}, \text{bp}.\mathbf{q}_g)$ 
30:  $t_{\text{sim}} \leftarrow T_1 + T_2 + T_g + \tau_c$ 
31: if  $t_{\text{sim}} > t^*$  then return None
32: end if
33: return  $\{t^*, t_{\text{sim}}, \text{margin}=t^*-t_{\text{sim}}, T_1, T_2, T_g, \text{bp}.\mathbf{q}_{pg}, \text{bp}.\mathbf{q}_g, \text{traj}_1, \text{traj}_2\}$ 

```

6.8.3 Contractive MPC for Dynamic Grasping

Standard NMPC minimises a cost function over a receding horizon, which provides good reference-tracking performance but does not guarantee that the tracking error converges monotonically. For dynamic grasping this matters: the robot must reduce its joint-position error below a tolerance ε within a fixed number of steps before the ball arrives. A pure cost-minimising controller may oscillate or converge too slowly under actuation limits.

To address this, an explicit per-step contraction constraint is added to the OCP that forces the joint-position error to shrink by a factor of at most α at every MPC step — providing a provable upper bound on the number of steps needed to reach ε regardless of trajectory cost.

At each stage k of the horizon the following inequality is appended to the OCP:

$$\|\mathbf{q}(k) - \mathbf{q}_{\text{ref}}\| \leq \alpha \|\mathbf{q}(k-1) - \mathbf{q}_{\text{ref}}\|. \quad (6.3)$$

Applying this recursively from the current error $\|\mathbf{e}_0\|$ gives the convergence guarantee:

$$\|\mathbf{e}_k\| \leq \alpha^k \|\mathbf{e}_0\| \implies \text{reach } \varepsilon \text{ in at most } \left\lceil \frac{\ln(\varepsilon / \|\mathbf{e}_0\|)}{\ln \alpha} \right\rceil \text{ steps.} \quad (6.4)$$

The factor α cannot be set arbitrarily small; too aggressive a rate makes the constraint infeasible given the robot's finite velocity and torque capacity. It is computed adaptively at each control step from two quantities:

$$\alpha_{\min} = 1 - \frac{\min_j(\dot{q}_{\max,j} h)}{\|\mathbf{e}\|}, \quad (6.5)$$

$$\alpha_{\text{req}} = \left(\frac{\varepsilon}{\|\mathbf{e}\|} \right)^{1/N}, \quad (6.6)$$

where $\alpha_{\min}$ is the physical feasibility lower bound (the contracted reference must not require a one-step displacement exceeding the joint velocity limit), and α_{req} is the rate needed to drive the error to ε within exactly N steps (returns 0 when already inside the basin $\|\mathbf{e}\| \leq \varepsilon$). The final contraction factor is:

$$\alpha = \max(\alpha_{\text{req}}, \alpha_{\min}). \quad (6.7)$$

The solver parameter vector is extended to $\mathbf{p} = [\mathbf{x}_{\text{ref}}^\top \text{ (14 elements); } \alpha \text{ (1 element)}]$ and the contraction inequality is added as a nonlinear constraint per stage. Both are updated online at every control step.

Rather than scanning a range of candidate times, the intercept point is fixed at $T_{\text{catch}} + T_{\text{grip}}$ seconds ahead of the current instant, where $T_{\text{catch}} = N \cdot h$ is the MPC lookahead horizon. The candidate is accepted only if: (i) the predicted ball position lies within the Panda's reachable workspace; (ii) IK converges to a valid configuration $\mathbf{q}_{\text{int}}$; and (iii) $\alpha_{\text{req}} \geq \alpha_{\min}$, confirming the contraction rate is physically achievable.

Algorithm 5 Contractive MPC Dynamic Grasping Loop

- 1: Initialise solver; open gripper
 - 2: **loop** every MPC step
 - 3: Read joint state $(\mathbf{q}, \dot{\mathbf{q}})$ and ball state $(\mathbf{p}_b, \mathbf{v}_b)$
 - 4: Predict intercept position from ball state; solve IK to get $\mathbf{q}_{\text{ref}}$
 - 5: Compute contraction factor $\alpha \leftarrow \max(\alpha_{\text{req}}, \alpha_{\min})$
 - 6: Solve OCP with contraction constraint; apply resulting torque
 - 7: **if** arm sufficiently close to intercept pose **and** ball arriving **then**
 - 8: Hold arm at $\mathbf{q}_{\text{ref}}$; close gripper; **exit**
 - 9: **end if**
 - 10: **end loop**
-

6.9 Dynamic Grasping Method Comparison

6.9.1 Experimental Setup

Three methods were benchmarked: (i) Feasible Intercept Search (brute-force quintic scan + ACADOS MPC); (ii) CROFT (bracketed root-finding + TOPP-RA + ACADOS MPC); (iii) Contractive MPC (fully online, no offline planning). Trials: 200 ball launches filtered to those entering the Panda’s reachable workspace within 15 s.

6.9.2 Results

Table 6.8: Dynamic grasping method comparison (averages over successful trials).

Metric	Intercept Search	CROFT	Contractive MPC
Success rate (%)	89.8	92.5	76.2
Catch time (s)	3.528	3.418	2.520
Search time (s)	0.159	0.164	–
Torque RMS (N·m)	18.58	17.70	21.51
EE path length (m)	1.699	1.651	1.877
Jerk RMS (m/s ³)	363.5	410.8	1032.1
Grasp distance error (mm)	3.22	3.14	3.85

CROFT achieves the highest success rate (92.5%) with the smallest grasp-distance error (3.14 mm). Contractive MPC achieves the fastest catch time (2.52 s) and requires no offline search, but it has a lower success rate (76.2%) and significantly higher jerk.

6.10 Final Enhancement: Ball-Tracking During Closure

CROFT-Tracking is an enhancement to the base CROFT pipeline that actively compensates for ball motion throughout the gripper closure phase. Rather than treating the intercept pose as fixed once the plan is committed, the method continuously re-computes the target grasp pose from the ball’s real-time position and velocity at every control step, using a one-step lookahead to predict the ball’s position one timestep ahead and commanding the arm to track it accordingly.

Grasping is triggered by a stability criterion: the arm is considered stably locked onto the moving target when the end-effector XY position remains within 1 mm of the ball’s XY position for five consecutive control steps. Once this condition is satisfied, the system transitions to a graduated closure phase in which the gripper fingers close

progressively over a predefined duration while the arm continues tracking the ball, ensuring correct finger alignment around the object before any gripping force is applied. In the final phase, the grip force is ramped up monotonically until the measured contact normal force on each finger exceeds the friction-based minimum force threshold $F_{\min}$, confirming that a secure and stable grasp of the moving object has been achieved.

Table 6.9: CROFT-Base vs. CROFT-Tracking over 200 trials.

Metric	CROFT-Base	CROFT-Tracking	Change
Success rate	92.5% (185/200)	68.8% (138/200)	−23.7 pp
Grasp distance error (mm)	3.13	0.462	−85.2%

CROFT-Tracking reduces the grasp distance error by 85% through continuous IK resolve during closure, but reduces the success rate by 23.7 percentage points. The scale of error in both CROFT-Tracking and CROFT-Base is small; the reduction achieved by CROFT-Tracking therefore appears marginal for the 30 mm sphere used in these experiments. However, for grasping tiny objects (radius $\lesssim 5$ mm) where sub-millimetre alignment is required, this reduction will be highly valuable and CROFT-Tracking would be the preferred variant.

Conclusion and Recommendations

Conclusion

This project developed and validated a complete software stack for dynamic object grasping with the Franka Emika Panda 7-DoF manipulator.

Modelling. The kinematic and dynamic model was derived analytically using the screw-theory framework, covering space- and body-frame Jacobians, screw matrices, and the Newton–Euler formulation with twists and wrenches. Parameters were extracted from the URDF and validated against MuJoCo simulation ground truth. The Python-based implementation proved computationally insufficient for real-time control, so most runtime kinematics and dynamics calculations were replaced by Pinocchio, enabling closed-loop model-based control at the required timestep.

Inverse kinematics. The analytical closed-form IK solver is more than $22\times$ faster than the Pinocchio-DLS numerical solver at the 99% success threshold (0.441 ms vs. 9.737 ms). A seed count of $n = 8$ balances success rate ($> 97.5\%$) and computation time (< 0.2 ms).

Trajectory planning. TOPP-RA is the only profile that enforces torque limits explicitly and produces provably time-optimal trajectories. It consistently yields lower tracking RMSE and energy than the quintic profile across all controllers.

Control. MPC achieves the best tracking accuracy and lowest total energy but requires 3.4–3.7 ms per step; SMC provides the best disturbance rejection and evaluates in 0.12–0.14 ms; feedback linearisation evaluates in < 0.07 ms but produces the highest tracking errors. The choice of trajectory planner has as much impact on closed-loop performance as the controller itself.

Dynamic grasping. CROFT achieves the best overall performance: 92.5% success rate and 3.14 mm grasp-distance error, with a planning time of ≈ 0.16 s. Contractive MPC offers a fully online alternative (zero planning time, 2.52 s catch time), but with a lower success rate (76.2%) and significantly higher jerk.

Recommendations

1. **Per-joint gain scheduling for FL.** The feedback linearisation controller uses a single set of gains across all joints, yet each joint has a different inertia, velocity limit, and displacement magnitude. Per-joint gains tuned to each joint’s inertia and expected trajectory speed would significantly close the RMSE gap between FL and MPC without the computational overhead of MPC.
2. **Jerk-bounded TOPP-RA.** The current TOPP-RA parametrisation enforces joint torque and velocity limits but places no bound on jerk. Extending it with explicit jerk constraints would produce smoother reference trajectories, reduce mechanical wear, and narrow the jerk advantage currently held by the quintic profile.
3. **CROFT-Tracking for small objects.** CROFT-Tracking re-solves IK continuously during gripper closure, reducing grasp distance error by 85%. For objects with radius $\lesssim 5$ mm where sub-millimetre alignment is critical, limiting ball speed to ≈ 0.4 m/s at the moment of closure would allow CROFT-Tracking to deliver its alignment improvement without triggering the latency failure mode observed at higher speeds.
4. **Improving the success rate of Contractive MPC.** Contractive MPC achieved only 76.2% success compared to 92.5% for CROFT, primarily because the fixed horizon lookahead can lock onto an infeasible intercept under fast-moving or curved ball trajectories. Incorporating an adaptive horizon length that increases when the current intercept candidate is marginal, or coupling Contractive MPC with a brief CROFT pre-pass to seed the intercept estimate, would improve reliability while retaining the online nature of the method.
5. **Nonlinear ball trajectory prediction.** All three dynamic grasping methods assume a linear (constant-velocity) ball trajectory. Real thrown or rolling objects are subject to gravity, floor friction, and spin, producing curved paths that grow more inaccurate the further ahead the prediction horizon extends. Integrating a physics-based prediction model (e.g., incorporating rolling resistance and gravitational deceleration) would improve intercept accuracy, particularly for fast or long-range balls.
6. **Deployment on real hardware.** All experiments were conducted in the MuJoCo simulator. Transferring the pipeline to a physical Franka Emika Panda requires addressing unmodelled joint flexibility, communication latency between the controller and the robot interface, and sensor noise in the ball-tracking system. The modular design of the software stack—with Pinocchio handling all real-time dynamics and the MPC running on a separate thread—makes this transition straightforward, and hardware validation is the logical next step.

References

- [1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [2] J. Carpentier et al., “The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives,” in *IEEE International Symposium on System Integration (SII)*, 2019, pp. 614–619.
- [3] R. Verschueren et al., “*acados*—a modular open-source framework for fast embedded optimal control,” *Mathematical Programming Computation*, vol. 14, no. 1, pp. 147–183, 2022.
- [4] H. Pham and Q.-C. Pham, “A new approach to time-optimal path parameterization based on reachability analysis,” *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 645–659, 2018.
- [5] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991.
- [6] V. I. Utkin, *Sliding Modes in Control and Optimization*. Springer, 1992.
- [7] Y. Nakamura and H. Hanafusa, “Inverse kinematic solutions with singularity robustness for robot manipulator control,” *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.
- [8] D. E. Whitney, “Resolved motion rate control of manipulators and human prostheses,” *IEEE Transactions on Man-Machine Systems*, vol. 10, no. 2, pp. 47–53, 1969.
- [9] E. W. Aboaf, S. M. Drucker, and C. G. Atkeson, “Task-level robot learning: ball throwing,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 1989, pp. 295–301.
- [10] B. Kim and A. Billard, “Catching objects in flight,” *IEEE Transactions on Robotics*, vol. 30, no. 5, pp. 1049–1065, 2014.
- [11] S. S. M. Salehian, M. Khoramshahi, and A. Billard, “A dynamical system approach for softly catching a flying object: Theory and experiment,” *IEEE Transactions on Robotics*, vol. 32, no. 2, pp. 462–471, 2016.

-
- [12] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, “TossingBot: Learning to throw arbitrary objects with residual physics,” *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020.
 - [13] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
 - [14] Franka Emika GmbH, *Panda Research Robot: Technical Documentation*. Munich, Germany, 2019.